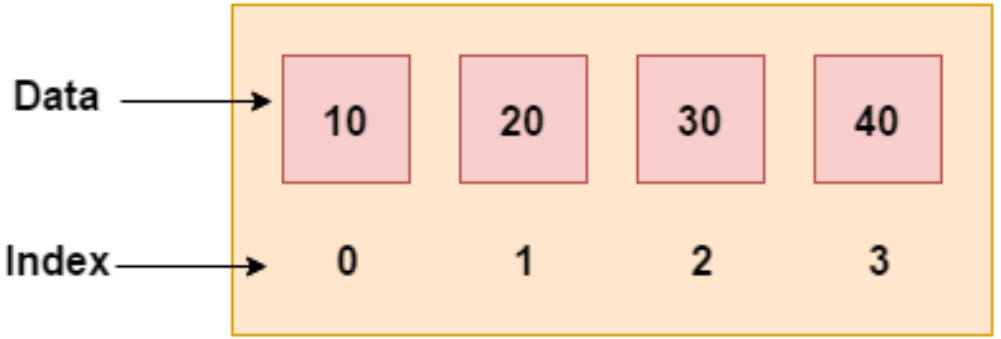
Object Oriented Programming(C++)
(CSE 1211)

C++: Arrays

Md. Mahfuzur Rahman
Lecturer,
Prime University

C++: Arrays

array in C++ is a group of similar types of elements that have contiguous memory location.



Advantages of C++ Array

- Code Optimization (less code)
- Random Access
- Easy to traverse data
- Easy to manipulate data
- Easy to sort data etc.

Disadvantages of C++ Array

- Fixed size

C++ Array Types

There are 2 types of arrays in C++ programming:

1.Single/One Dimensional Array

2.Multidimensional Array

C++ Single Dimensional Array

```
#include <iostream>
using namespace std;
int main()
{
 int arr[5]={10, 0, 20, 0, 30};  //creating and initializing array
        //traversing array
        for (int i = 0; i < 5; i++)
        {
            cout<<arr[i]<<"\n";
        }
}
```

Array

An **array** is a collection of elements of the same type placed in contiguous memory locations. It allows you to store multiple values under a single name and access them using an index.

We can create/declare an array by simply specifying the data type first and then the name of the array with its size inside **[] square brackets(better** known as **array subscript operator**).

data_type array_name [size]

This statement will create an array with name **array_name** that can store **size** elements of given **data_type.** Once the array is declared, its **size cannot be changed.**

int arr[5];

Initialization means assigning initial values to array elements. We can initialize the array with values enclosed in **curly braces '{}'** are assigned to the array. **For example:**

```
int arr[5] = {2, 4, 8, 12, 16};
```

Array

int x[6];

Here,

int - type of element to be stored

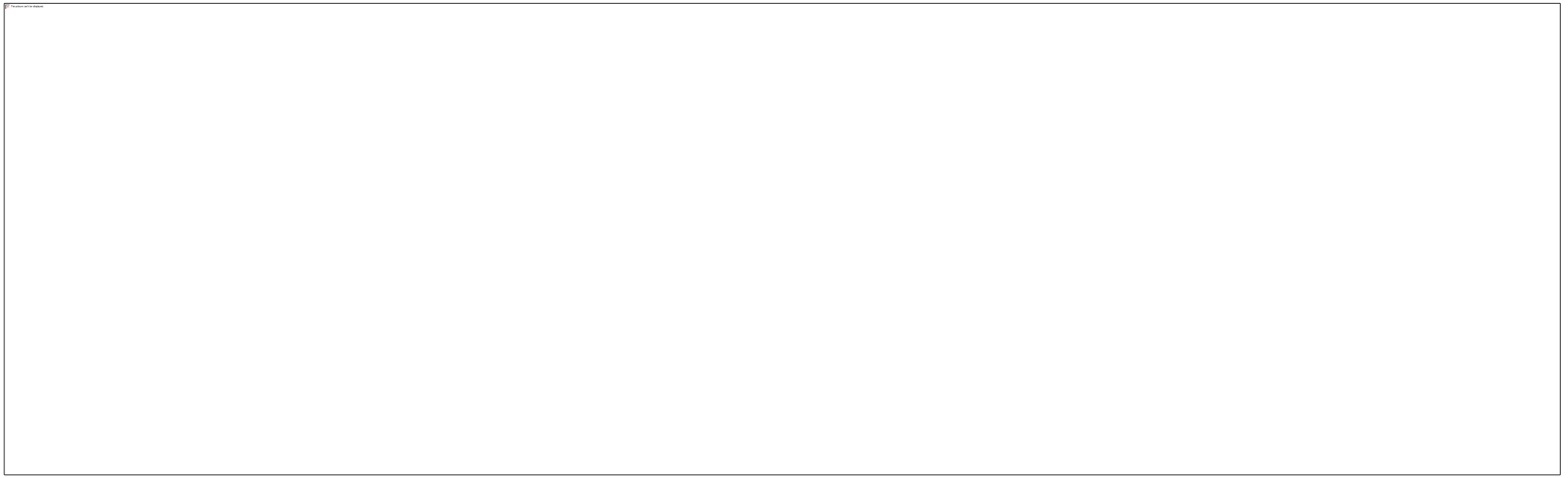
x - name of the array

6 - size of the array



Array

```
// store only 3 elements in the array
int x[6] = {19, 10, 8};
```



Array

```
string cars[4] = {"Volvo", "BMW", "Ford", "Mazda"};
cout << cars[0];
// Outputs Volvo
```

```
string cars[4] = {"Volvo", "BMW", "Ford", "Mazda"};
cars[0] = "Opel";
cout << cars[0];
// Now outputs Opel instead of Volvo
```

Array

```cpp
#include <iostream>
using namespace std;

int main() {

  int numbers[5] = {7, 5, 6, 12, 35};

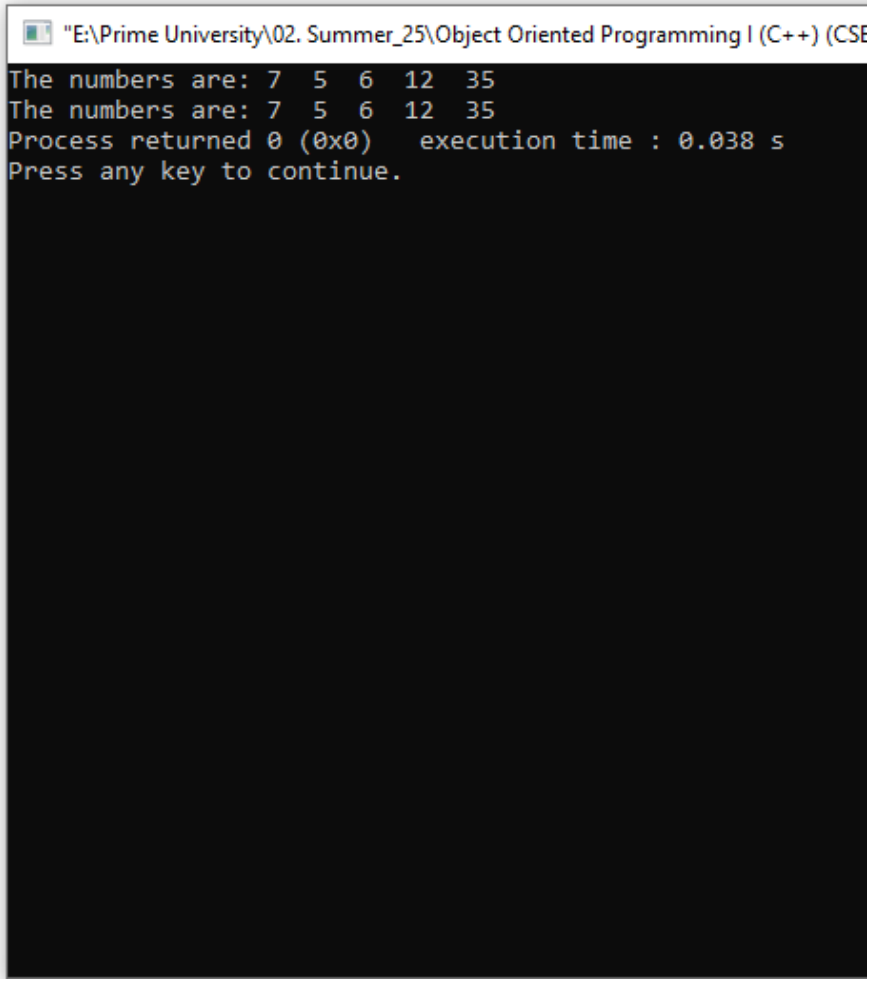
  cout << "The numbers are: ";

  //  Printing array elements
  // using range based for loop
  for (int n : numbers) {
    cout << n << "  ";
  }

  cout << "\nThe numbers are: ";

  //  Printing array elements
  // using traditional for loop
  for (int i = 0; i < 5; ++i) {
    cout << numbers[i] << "  ";
  }

  return 0;
}
```



Array

```cpp
#include <iostream>
using namespace std;

int main() {

  int numbers[5];

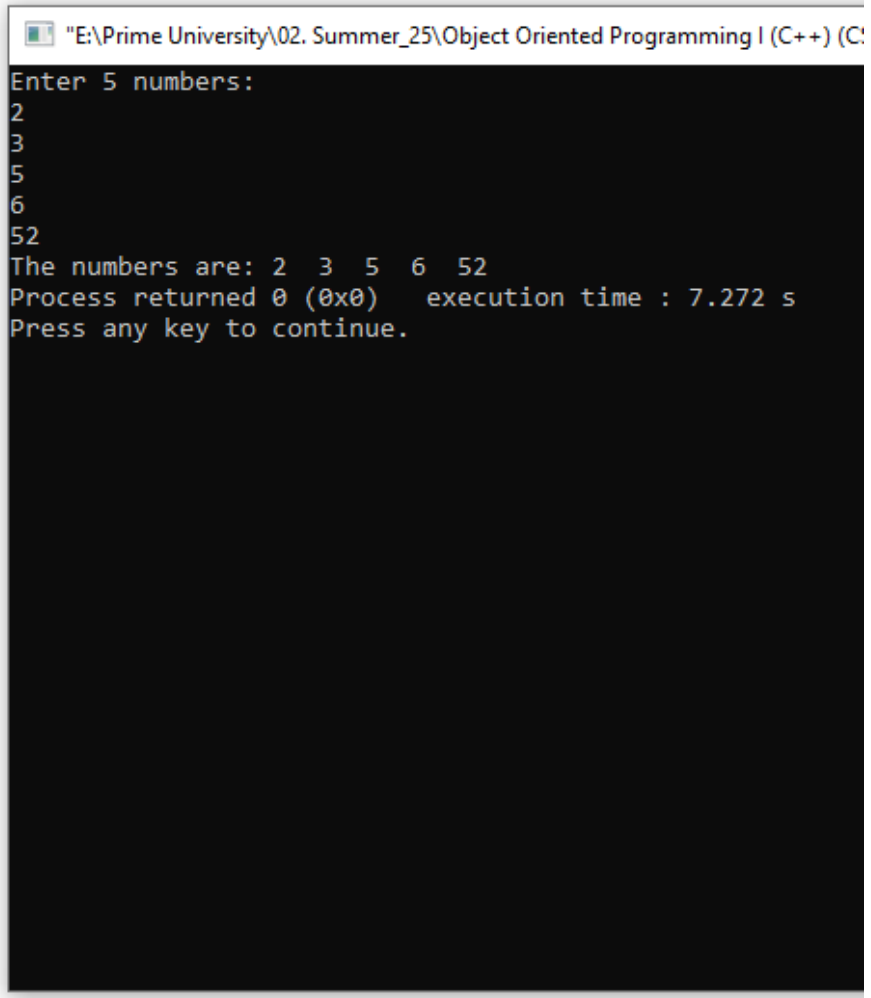
  cout << "Enter 5 numbers: " << endl;

  //  store input from user to array
  for (int i = 0; i < 5; ++i) {
    cin >> numbers[i];
  }

  cout << "The numbers are: ";

  //  print array elements
  for (int n = 0; n < 5; ++n) {
    cout << numbers[n] << "  ";
  }

  return 0;
}
```



Array

```cpp
int main() {

  // initialize an array without specifying size
  double numbers[] = {7, 5, 6, 12, 35, 27};

  double sum = 0;
  double count = 0;
  double average;

  cout << "The numbers are: ";

  //  print array elements
  // use of range-based for loop
  for (const double &n : numbers) {
    cout << n << "  ";

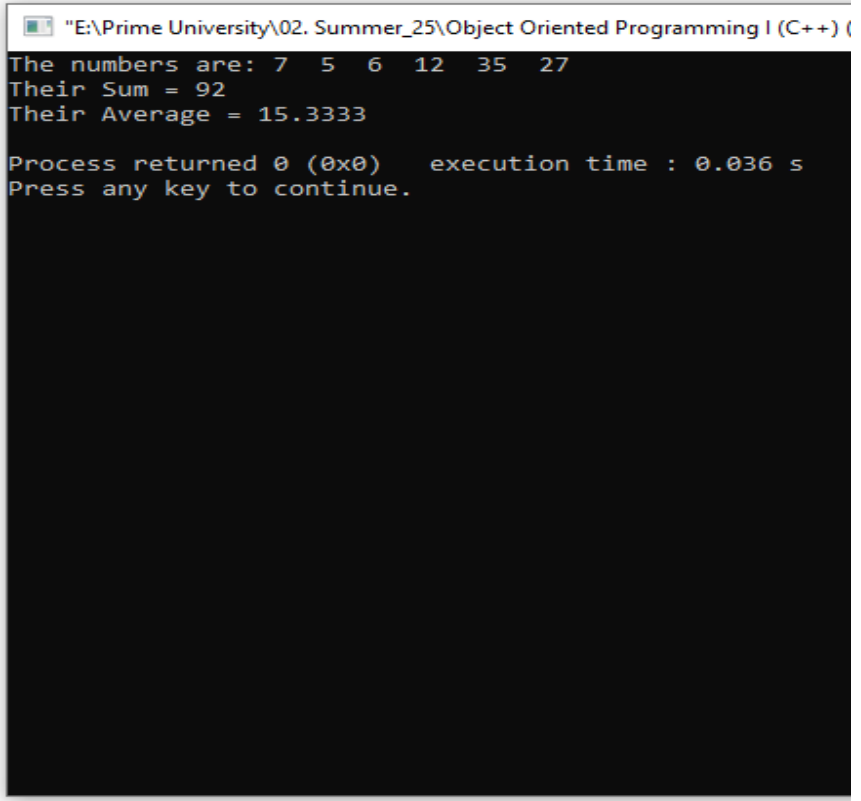
    //  calculate the sum
    sum += n;

    // count the no. of array elements
    ++count;
  }

  // print the sum
  cout << "\nTheir Sum = " << sum << endl;

  // find the average
  average = sum / count;
  cout << "Their Average = " << average << endl;

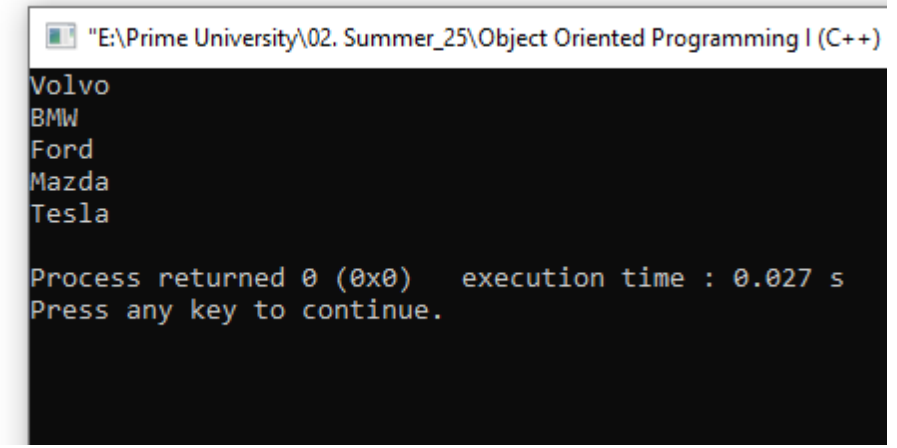
  return 0;
}
```



Arrays and Loops

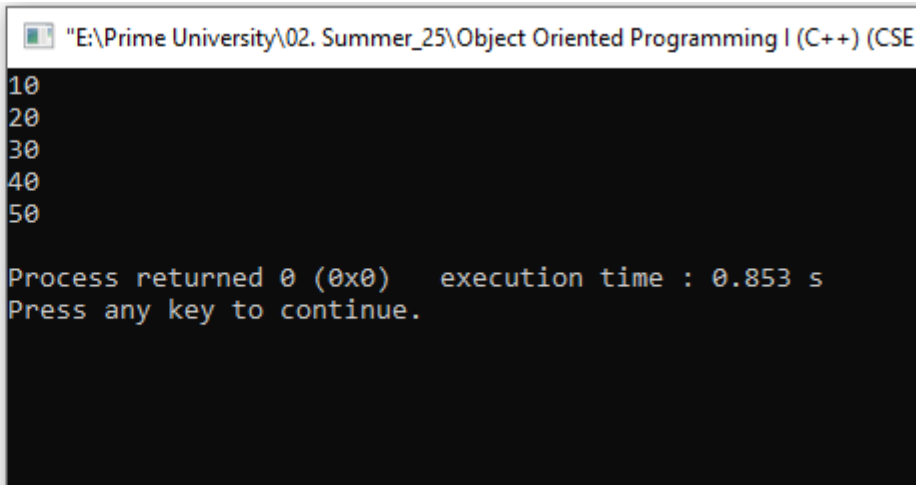
```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
  string cars[5] = {"Volvo", "BMW", "Ford", "Mazda", "Tesla"};
  for (int i = 0; i < 5; i++) {
    cout << cars[i] << "\n";
  }
  return 0;
}
```




```cpp
#include <iostream>
using namespace std;

int main() {
  int myNumbers[5] = {10, 20, 30, 40, 50};
  for (int i = 0; i < 5; i++) {
    cout << myNumbers[i] << "\n";
  }
  return 0;
}
```



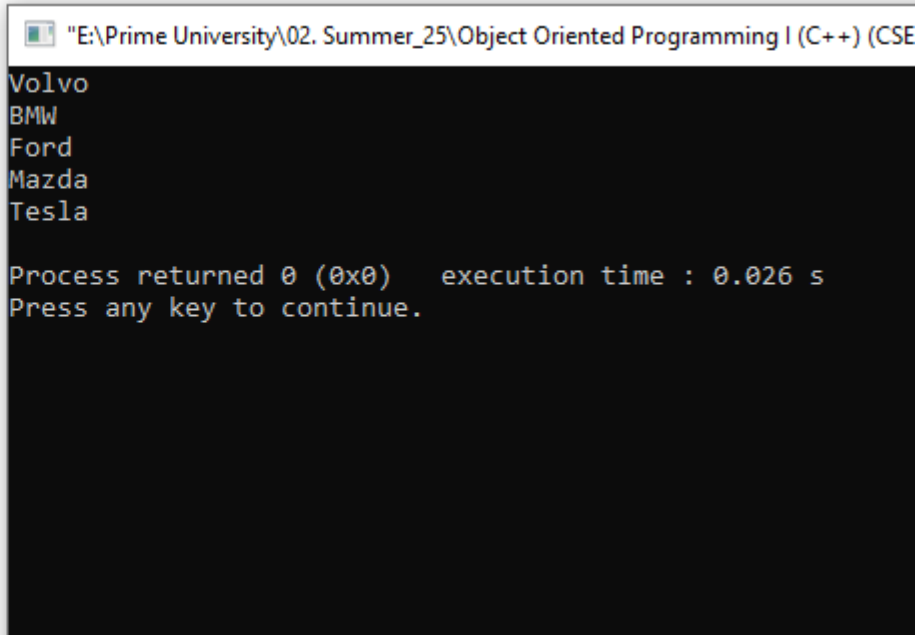
Arrays and Loops (foreach Loop)

for (*type variableName* : *arrayName*) {
  // code block to be executed
}

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
  // Create an array of strings
  string cars[5] = {"Volvo", "BMW", "Ford", "Mazda", "Tesla"};

  // Loop through strings
  for (string car : cars) {
    cout << car << "\n";
  }
  return 0;
}
```



Arrays and Loops

```
int myNumbers[5] = {10, 20, 30, 40, 50};
cout << sizeof(myNumbers);
```

Output: 20

C++ Array Example: Traversal using foreach loop

We can also traverse the array elements using foreach loop. It returns array element one by one.

```
#include <iostream>
using namespace std;
int main()
{
 int arr[5]={10, 0, 20, 0, 30}; //creating and initializing array
       //traversing array
      for (int i: arr)
       {
          cout<<i<<"\n";
       }
}
```

Why do we need arrays?

With a limited number of objects, we can use regular variables (v1, v2, v3,..), but when we need to hold many instances, managing them with normal variables becomes challenging. To represent numerous instances in one variable, we use an array.

What happens if we try to access out of bound array?

The array's elements will be numbered 0 through 9 if we define an array of size 10.

We will have Undefined Behavior if we attempt to access an element at an index higher than 10, though.

C++ array with empty members

The maximum number of elements that can be stored in an array in C++ is n. What will happen, though, if we store fewer than n elements?

For example,

```
// store only 3 elements in the array
int x[6] = {19, 10, 8};
```

The array x in this case is 6 elements wide. But we've only given it a three-element initialization.
When that happens, the compiler fills in the empty spaces with random values. This random value frequently appears as 0.

Important things to remember while using arrays in C++

1) The array's indexes begin at 0. Meaning that the first item saved at index 0 is x[0].
2) The final element of an array with size n is kept at index (n-1). This example's final element is x[5].
3) An array's elements have sequential addresses. Consider the scenario where x[0] beginning 's address is 2120. The address of the subsequent element, x[1], will then be 2124, followed by x[2], 2128, and so forth.

Each element in this case has a four-fold increase in size. This is due to the fact that int has a 4 byte capacity.

How to display the sum and average of array elements?

```
#include <iostream>
using namespace std;
int main() {
int n, sum=0;
cout << “Enter Number of Students: ";
cin>>n;
 int students[n];
  for (int i=0; i<n; i++) {
  cin>>students[i];
  sum=sum+students[i];
}
 // print the sum
cout << "\n Their Sum = " << sum << endl;
// find the average
float average = (float)sum / n;
cout << "Their Average = " << average << endl;
  return 0;
}
```

C++ Passing Array to Function

In C++, to reuse the array logic, we can create function. To pass array to function in C++, we need to provide only array name.

functionname(arrayname); //passing array to function

C++ Passing Array to Function Example: print array elements

```
#include <iostream>
using namespace std;
void printArray(int arr[5]);
int main()
{
        int arr1[5] = { 10, 20, 30, 40, 50 };
        int arr2[5] = { 5, 15, 25, 35, 45 };
        printArray(arr1); //passing array to function
        printArray(arr2);
}
```

```
void printArray(int arr[5])
{
    cout << "Printing array elements:"<< endl;
    for (int i = 0; i < 5; i++)
    {
                cout<<arr[i]<<"\n";
    }
}
```

C++ Passing Array to Function Example: Print minimum number

```
#include <iostream>
using namespace std;
void  printMin(int arr[5]);
int main()
{
   int arr1[5] = { 30, 10, 20, 40, 50 };
        int arr2[5] = { 5, 15, 25, 35, 45 };
        printMin(arr1);//passing array to function
         printMin(arr2);
}
```

```
void  printMin(int arr[5])
{
    int min = arr[0];
        for (int i = 0; i > 5; i++)
        {
            if (min > arr[i])
            {
                min = arr[i];
            }
        }
        cout<< "Minimum element is: "<< min <<"\n";


}
```

C++ Passing Array to Function Example: Print maximum number

```
#include <iostream>
using namespace std;
void  printMax(int arr[5]);
int main()
{
        int arr1[5] = { 25, 10, 54, 15, 40 };
        int arr2[5] = { 12, 23, 44, 67, 54 };
        printMax(arr1); //Passing array to function
         printMax(arr2);
}
```

```
void  printMax(int arr[5])
{
    int max = arr[0];
        for (int i = 0; i < 5; i++)
        {
            if (max < arr[i])
            {
                max = arr[i];
            }
        }
        cout<< "Maximum element is: "<< max <<"\n";
}
```

Two-Dimensional Array in C++

A **two-dimensional array** or **2D array** in C++ is the simplest form of the multidimensional array. We can visualize a two-dimensional array as an array of one-dimensional arrays arranged one over another forming a table with ‘x’ rows and ‘y’ columns where the row number ranges from 0 to (x-1) and the column number ranges from 0 to (y-1).

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]
Row 2	x[2][0]	x[2][1]	x[2][2]

Declaration of Two-Dimensional Array in C++

The basic form of declaring a 2D array with **x** rows and **y** columns in C++ is shown below.

Syntax:

data_type array_name[x][y];

where,

- **data_type:** Type of data to be stored in each element.
- **array_name:** name of the array
- **x:** Number of rows.
- **y:** Number of columns.

Initialization of Two-Dimensional Arrays in C++

The various ways in which a 2D array can be initialized are as follows:

1.Using Initializer List

2.Using Loops

1. Initialization of 2D array using Initializer List

We can initialize a 2D array in C by using an initializer list as shown in the example below.

First Method:

int x[3][4] = {0, 1 ,2 ,3 ,4 , 5 , 6 , 7 , 8 , 9 , 10 , 11}

Second Method (better):

int x[3][4] = {{0,1,2,3}, {4,5,6,7}, {8,9,10,11}};

2. Initialization of 2D array using Loops

We can use any C loop to initialize each member of a 2D array one by one as shown in the below example.

Example:

```
int x[3][4];
for(int i = 0; i < 3; i++)
  { for(int j = 0; j < 4; j++)
    {
     x[i][j] = i + j;
     }
  }
```

This method is useful when the values of each element have some sequential relation.

C++ Two dimensional Array Example

```
#include <iostream>
using namespace std;
int main()
{
  int test[3][3];  //declaration of 2D array
    test[0][0]=5;  //initialization
    test[0][1]=10;
    test[1][1]=15;
    test[1][2]=20;
    test[2][0]=30;
    test[2][2]=10;
```

```
//traversal
    for(int i = 0; i < 3; ++i)
    {
        for(int j = 0; j < 3; ++j)
        {
            cout<< test[i][j]<<" ";
        }
        cout<<endl;
    }
    return 0;
}
```

C++ Multidimensional Arrays

The multidimensional array is also known as rectangular arrays in C++. It can be two dimensional or three dimensional. The data is stored in tabular form (row * column) which is also known as matrix.

A multi-dimensional array can be termed as an array of arrays that stores homogeneous data in tabular form. Data in multidimensional arrays is generally stored in row-major order in the memory.

The ***general form of declaring N-dimensional arrays*** is shown below.

Syntax:

data_type array_name[size1][size2]....[sizeN];

- **data_type**: Type of data to be stored in the array.
- **array_name**: Name of the array.
- **size1, size2,…, sizeN**: Size of each dimension.

Examples:

Two dimensional array: int two_d[10][20];

Three dimensional array: int three_d[10][20][30];

Size of Multidimensional Arrays:

The total number of elements that can be stored in a multidimensional array can be calculated by multiplying the size of all the dimensions.

For example:

- The array **int x[10][20]** can store total (10*20) = 200 elements.
- Similarly array **int x[5][10][20]** can store total (5*10*20) = 1000 elements.

To get the size of the array in bytes, we multiply the size of a single element with the total number of elements in the array.

For example:

- Size of array **int x[10][20]** = 10 * 20 * 4 = 800 bytes. (where int = 4 bytes)
- Similarly, size of **int x[5][10][20]** = 5 * 10 * 20 * 4 = 4000 bytes. (where int = 4 bytes)

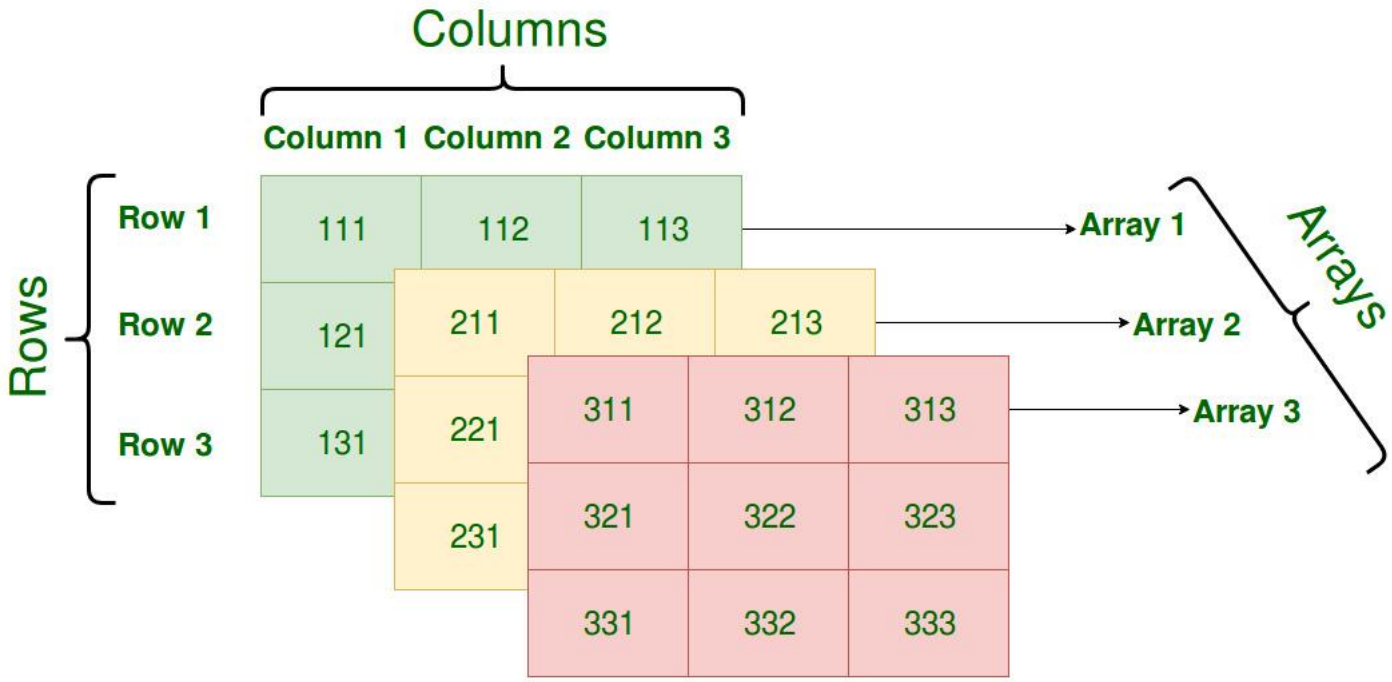
C++ Multidimensional Array Example: Declaration and initialization at same time

```
#include <iostream>
using namespace std;
int main()
{
  int test[3][3] =
    {
        {2, 5, 5},
        {4, 0, 3},
        {9, 1, 8}  };  //declaration and initialization

//traversal
    for(int i = 0; i < 3; ++i)
    {
        for(int j = 0; j < 3; ++j)
        {
            cout<< test[i][j]<<" ";
        }
        cout<<"\n"; //new line at each row
    }
    return 0;
}
```

Three-Dimensional Array in C++

A **Three Dimensional Array** or **3D** array in C++ is a collection of two-dimensional arrays. It can be visualized as multiple 2D arrays



Declaration of Three-Dimensional Array in C++

We can declare a 3D array with **x** 2D arrays each having **y** rows and **z** columns using the syntax shown below.

Syntax:

data_type array_name[x][y][z];

- **data_type:** Type of data to be stored in each element.
- **array_name:** name of the array
- **x:** Number of 2D arrays.
- **y:** Number of rows in each 2D array.
- **z:** Number of columns in each 2D array.

Example:

```
int array[3][3][3];
```

Initialization of Three-Dimensional Array in C++

Initialization in a 3D array is the same as that of 2D arrays. The difference is as the number of dimensions increases so the number of nested braces will also increase.
A 3D array in C++ can be initialized by using:

1.Initializer List
2.Loops

Initialization of 3D Array using Initializer List

Method 1:

```
int x[2][3][4] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23};
```

Method 2(Better):

```
int x[2][3][4] = { { {0,1,2,3}, {4,5,6,7}, {8,9,10,11} }, { {12,13,14,15}, {16,17,18,19}, {20,21,22,23} } };
```

Initialization of 3D Array using Loops

It is also similar to that of 2D array with one more nested loop for accessing one more dimension.

```
int x[2][3][4];
for (int i=0; i<2; i++)
  { for (int j=0; j<3; j++)
     { for (int k=0; k<4; k++)
        {
         x[i][j][k] = (some_value);
         }
     }
  }
```

Fibonacci Number & Factorial Number

```c
#include<stdio.h>
int main()
{
    int first=0,second=1,m,n,i;

    scanf("%d",&n);

    for(i=1;i<=n;i++)
    {
         printf(" %d",first);
        m = first + second;
        first = second;
        second = m;

    }

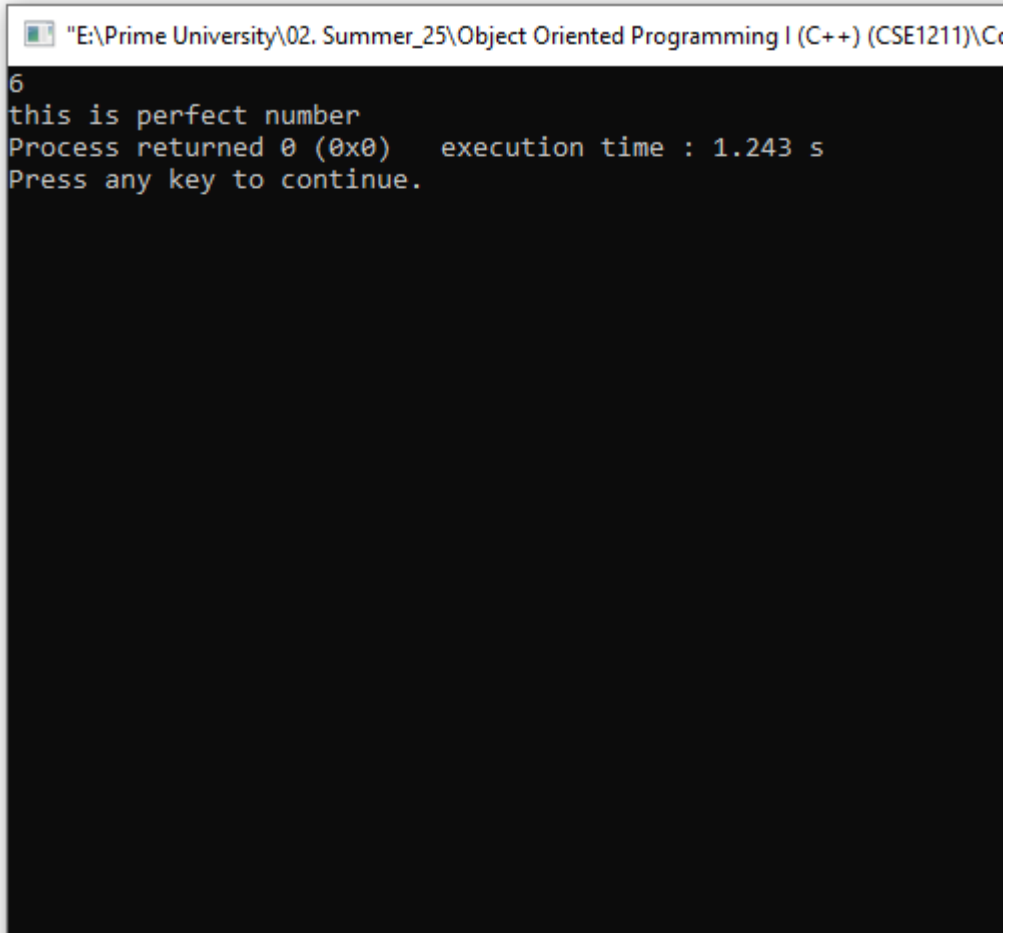
    return 0;
}
```

```c
#include<stdio.h>
int main()
{
   int i,n,fact=1;
    scanf("%d",&n);
    for(i=1;i<n;i++)
    {
        fact=fact*i;
    }
    printf(" the factorial number is;%d",fact);
    return 0;

}
```

Perfect Number

```c
#include<stdio.h>
int main()
{
    int num,i,sum=0;
    scanf("%d",&num);
    for(i=1;i<num;i++)
    {
        if(num%i==0)
            sum=sum+i;
    }
    if(num==sum)
    {
        printf("this is perfect number",num);
    }
    else
    {
        printf("this is not perfect number ",num);
    }
    return 0;
}
```



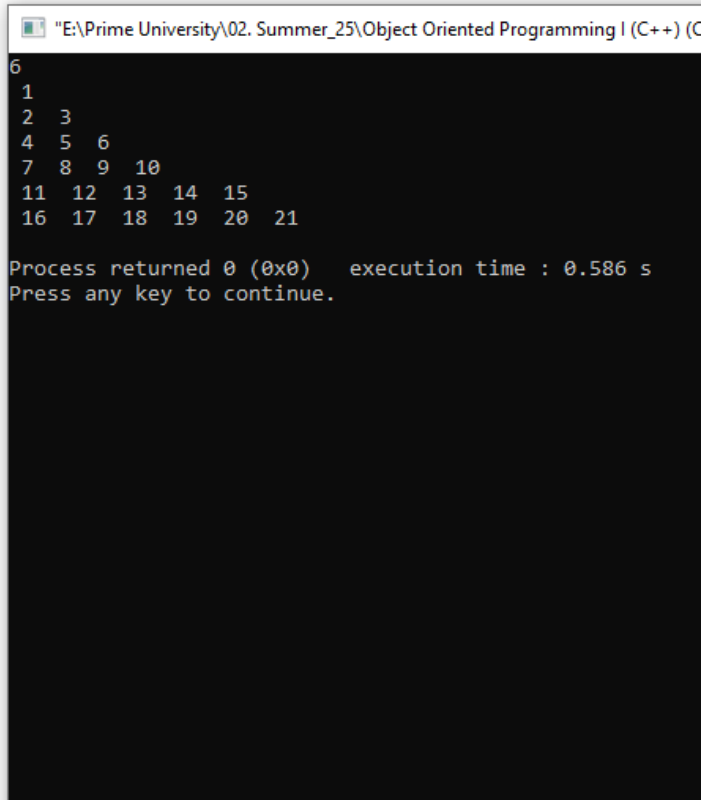
Number Print

```c
#include<stdio.h>
int main()
{
    int i,n,j,result=0;
    scanf("%d",&n);
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=i;j++)
        {
        result=result+1;

           printf(" %d ",result);
            //printf("*");

        }
        printf("\n");

    }
    return 0;
}
```



Prime Number

```c
#include<stdio.h>
int main()
{
      int i,num,  flag = 0;
      scanf("%d",&num);
      for(i=2;i<num;i++)
      {
           if(num%i==0)
           {
                flag = 1;
              break;
           }
      }

      if(flag == 0)
          {
              printf("number is prime");
          }
      else{
           printf("number is not prime");
          }

          return 0;
}
```

Strong Number

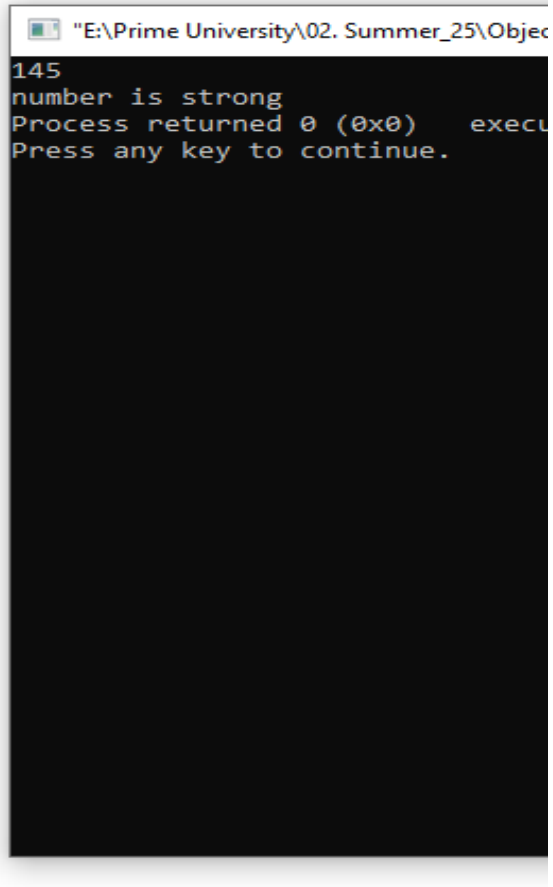
```c
#include<stdio.h>
int main()
{
    int num,i,r,sum=0,a,fact;
    scanf("%d",&num);
    a=num;
    while(num>0)
    {
        fact=1;
        r=num%10;


        for(i=1;i<=r;i++)
    {
        fact=fact*i;

    }
    sum=sum+fact;
    num=num/10;
    }
    if(sum==a)
        printf( "number is strong",a);
        else
            printf("number is not strong",a);

        return 0;

}
```



END

C++ Pointers

- The pointer in C++ language is a variable, it is also known as locator or indicator that points to an address of a value.
- The symbol of an address is represented by a pointer.
- In addition to creating and modifying dynamic data structures, they allow programs to emulate call-by-reference.
- One of the principal applications of pointers is iterating through the components of arrays or other data structures.
- The pointer variable that refers to the same data type as the variable you're dealing with has the address of that variable set to it (such as an int or string).

Syntax

```
datatype *var_name;
int *ptr;   // ptr can point to an address which holds int data
```

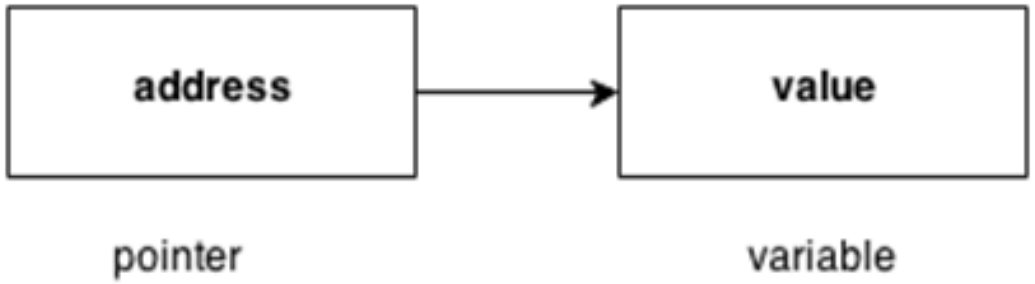
How to use a pointer?

1.Establish a pointer variable.

2.employing the unary operator (&), which yields the address of the variable, to assign a pointer to a variable's address.

3.Using the unary operator (*), which gives the variable's value at the address provided by its argument, one can access the value stored in an address.

Since the data type knows how many bytes the information is held in, we associate it with a reference. The size of the data type to which a pointer points is added when we increment a pointer.



Pointer

- int x = 5;
- int y = 5;

printf("value of x = %d\n",x);

printf("Address of x = %d\n",&x);

Pointers is a variable that stores/points the address of another variable.



Declaration of pointer

---

data_type *var_name;

Example : int *p; char *p;

Pointer

2 symbol

& symbol is used to get the address of the variable.

* symbol is used to get the value of the variable that the pointer is pointing to.




```
int x = 5;
int y = 10;
int *p;

p = &x;

printf("value of x = %d\n",x);
printf("Address of x = %d\n",&x);
printf("Address of p = %d\n",p);
printf("content  of p = %d\n",*p);
printf("Address of p variable = %d\n",&p);
```

Symbols used in pointer

Symbol	Name	Description
& (ampersand sign)	Address operator	Determine the address of a variable.
* (asterisk sign)	Indirection operator	Access the value of an address.

Declaring a pointer

The pointer in C++ language can be declared using * (asterisk symbol).

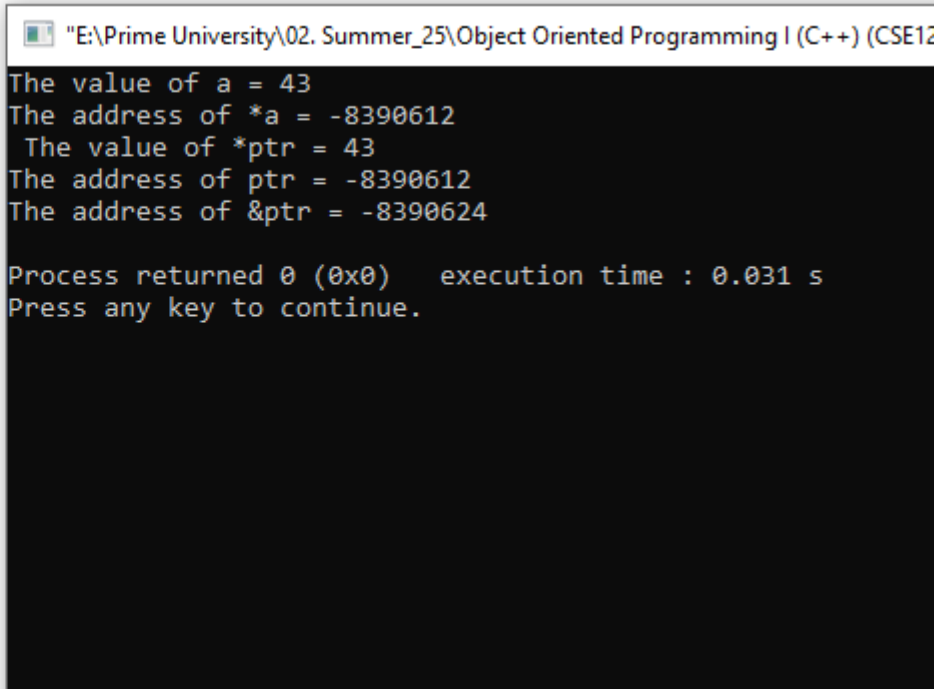
```
int *  a; //pointer to int
char *  c; //pointer to char
```

Pointer

```c
#include<stdio.h>
int main()
{
    int a = 43;
    int *ptr = &a;

    printf("The value of a = %d\n", a);
    printf("The address of *a = %d\n", &a);
    printf(" The value of *ptr = %d\n", *ptr);
    printf("The address of ptr = %d\n", ptr);
    printf("The address of &ptr = %d\n", &ptr);

    return 0;
}
```



Adding two numbers using pointer

```c
#include <stdio.h>
int main()
{
    int a, b, *ptr1, *ptr2, sum;

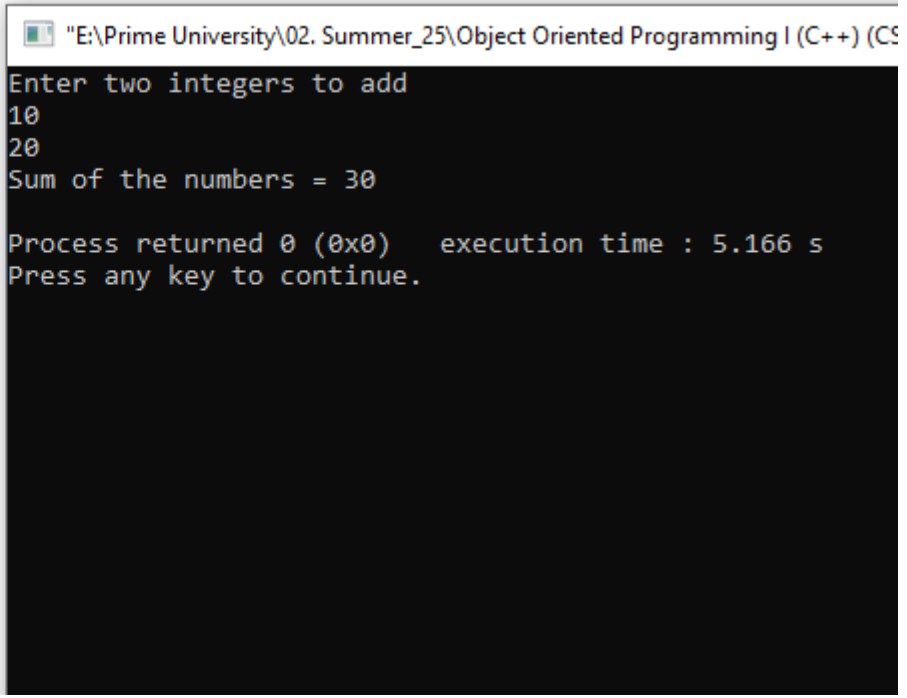
    printf("Enter two integers to add\n");
    scanf("%d%d", &a, &b);

    ptr1 = &a;
    ptr2 = &b;

    sum = *ptr1 + *ptr2;

    printf("Sum of the numbers = %d\n", sum);

    return 0;
}
```



Swap

```c
#include <stdio.h>

int main() {
    int a = 5, b = 10, temp;

    // Swapping values of a and  b
    temp = a;
    a = b;
    b = temp;
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

```c
#include <stdio.h>

int main() {
    int a = 5, b = 10;

    // Arithmetic operations to swap values
    a = a + b;
    b = a - b;
    a = a - b;
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

```cpp
#include <iostream>
using namespace std;

int main() {
    int a = 1, b = 55;
    cout << a << " " << b << endl;

    // swapping the values of a and b
    swap(a, b);

    cout << a << " " << b;
    return 0;
}
```

Swap using Pointer

```c
#include <stdio.h>
void swap(int *ptr1, int *ptr2) {
    int temp;
    temp = *ptr1;
    *ptr1 = *ptr2;
    *ptr2 = temp;
}

int main() {
    int num1 = 10;
    int num2 = 20;

    printf("Before swapping: num1 = %d, num2 = %d\n", num1, num2);
    swap(&num1, &num2);
    printf("After swapping: num1 = %d, num2 = %d\n", num1, num2);

    return 0;
}
```

Advantage of pointer

1) Pointer reduces the code and improves the performance, it is used to retrieving strings, trees etc. and used with arrays, structures and functions.

2) We can return multiple values from function using pointer.

3) It makes you able to access any memory location in the computer's memory.

Usage of pointer

There are many usage of pointers in C++ language.

1) Dynamic memory allocation

In c language, we can dynamically allocate memory using malloc() and calloc() functions where pointer is used.

2) Arrays, Functions and Structures

Pointers in c language are widely used in arrays, functions and structures. It reduces the code and improves the performance.

Pointer Example

```
#include <iostream>
using namespace std;
int main()
{
int number=30;
int *  p;
p=&number;//stores the address of number variable
cout<<"Address of number variable is:"<<&number<<endl;
cout<<"Address of p variable is:"<<p<<endl;
cout<<"Value of p variable is:"<<*p<<endl;
   return 0;
}
```

Pointer Program to swap 2 numbers without using 3rd variable

```
#include <iostream>
using namespace std;
int main()
{
int a=20,b=10,*p1=&a,*p2=&b;
cout<<"Before swap: *p1="<<*p1<<" *p2="<<*p2<<endl;
*p1=*p1+*p2;
*p2=*p1-*p2;
*p1=*p1-*p2;
cout<<"After swap: *p1="<<*p1<<" *p2="<<*p2<<endl;
   return 0;
}
```

What are Pointer and string literals?

String literals are arrays of character sequences with null ends. The elements of a string literal are arrays of type const char (because characters in a string cannot be modified) plus a terminating null-character.

What is a void pointer?

This unique type of pointer, which is available in C++, stands in for the lack of a kind. Pointers that point to a value that has no type are known as void pointers (and thus also an undetermined length and undetermined dereferencing properties). This indicates that void pointers are very flexible because they can point to any data type. This flexibility has benefits. Direct dereference is not possible with these pointers. Before they may be dereferenced, they must be converted into another pointer type that points to a specific data type.

What is a invalid pointer?

A pointer must point to a valid address, not necessarily to useful items (like for arrays). We refer to these as incorrect pointers. Additionally, incorrect pointers are uninitialized pointers.

```
int *ptr1;
int arr[10];
int *ptr2 = arr+20;
```

Here, ptr1 is not initialized, making it invalid, and ptr2 is outside of the bounds of arr, making it likewise weak. (Take note that not all build failures are caused by faulty references.)

What is a null pointer?

A null pointer is not merely an incorrect address; it also points nowhere. Here are two ways to mark a pointer as NULL:

```
int *ptr1 = 0;
int *ptr2 = NULL;
```

What is a pointer to a pointer?

In C++, we have the ability to build a pointer to another pointer, which might then point to data or another pointer. The unary operator (*) is all that is needed in the syntax for declaring the pointer for each level of indirection.

```
char a;
char *b;
char ** c;
a = 'g';
b = &a;
c = &b;
```

Here b points to a char that stores 'g', and c points to the pointer b.

What are references and pointers?

1.Call-By-Value

2.Call-By-Reference with a Pointer Argument

3.Call-By-Reference with a Reference Argument

Example

```
#include <iostream>
using namespace std;
// Pass-by-Value
int square1(int n)
{cout << "address of n1 in square1(): "
<< &n << "\n";
n *= n;
return n;
}
```

```
// Pass-by Reference with Pointer Arguments
void square2(int* n)
{
cout << "address of n2 in square2(): " << n << "\n"
*n *= *n;
}
```

```
// Pass-by-Reference with Reference Arguments
void square3(int& n)
{

cout << "address of n3 in square3(): " << &n << "\n";
n *= n;
}
void example()
{
   // Call-by-Value
   int n1 = 8;
   cout << "address of n1 in main(): " << &n1 << "\n";
   cout << "Square of n1: " << square1(n1) << "\n";
   cout << "No change in n1: " << n1 << "\n";
```

```
// Call-by-Reference with Pointer Arguments
   int n2 = 8;
   cout << "address of n2 in main(): " << &n2 << "\n";
   square2(&n2);
   cout << "Square of n2: " << n2 << "\n";
   cout << "Change reflected in n2: " << n2 << "\n";

   // Call-by-Reference with Reference Arguments
   int n3 = 8;
   cout << "address of n3 in main(): " << &n3 << "\n";
   square3(n3);
   cout << "Square of n3: " << n3 << "\n";
   cout << "Change reflected in n3: " << n3 << "\n";
}
// Driver program
int main() { example(); }
```

sizeof() operator in C++

The sizeof() is an operator that evaluates the size of data type, constants, variable. It is a compile-time operator as it returns the size of any variable or a constant at the compilation time.
The size, which is calculated by the sizeof() operator, is the amount of RAM occupied in the computer.

Syntax of the sizeof() operator is given below:

sizeof(data_type);

In the above syntax, the data_type can be the data type of the data, variables, constants, unions, structures, or any other user-defined data type.

The sizeof () operator can be applied to the following operand types:

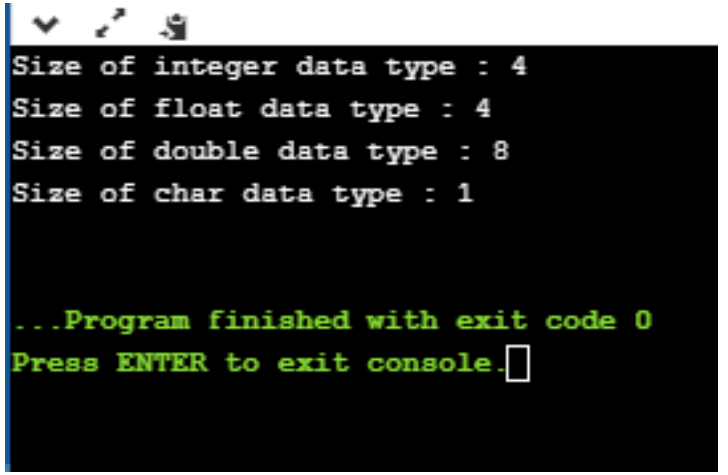
- **When an operand is of data type**

If the parameter of a **sizeof()** operator contains the data type of a variable, then the **sizeof()** operator will return the size of the data type.

Let's understand this scenario through an example.

```
#include <iostream>
using namespace std;
int main()
{
  // Determining the space in bytes occupied by each data type.
  cout << "Size of integer data type : " <<sizeof(int)<< endl;
  cout << "Size of float data type : " <<sizeof(float)<< endl;
  cout << "Size of double data type : " <<sizeof(double)<< endl;
  cout << "Size of char data type : " <<sizeof(char)<< endl;
  return 0;
}
```

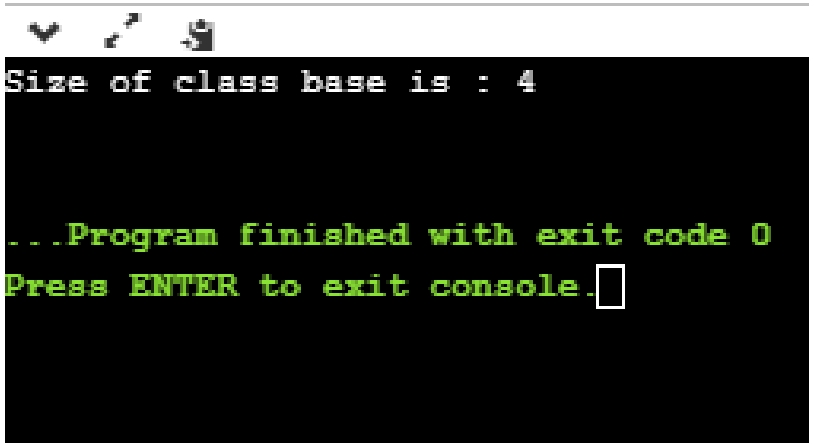
Output:



- **When an operand is of Class type.**

```
#include <iostream>
using namespace std;
class Base
{
int a;
};
int main()
{
Base b;
cout << "Size of class base is : "<<sizeof(b) << endl;
return 0;
}
```

Output:

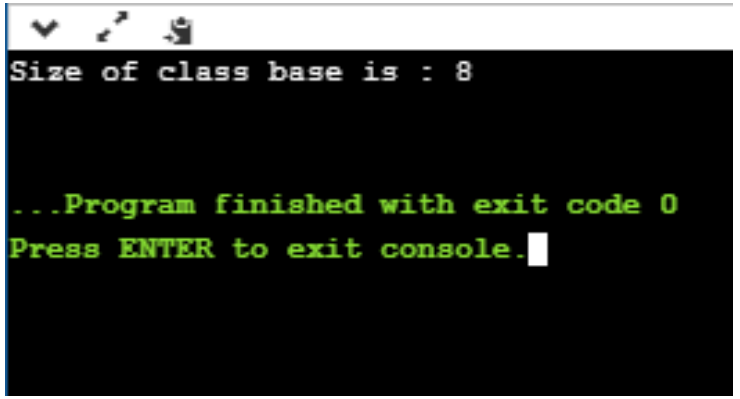


In the above program, we have evaluated the size of the class, which is having a single integer variable. The output would be 4 bytes as int variable occupies 4 bytes.

If we add one more integer variable in a class, then the code would look like:

```
#include <iostream>
using namespace std;
class Base
{
    int a;
    int d;
};
int main()
{
  Base b;
  cout << "Size of class base is : "<<sizeof(b) << endl;
  return 0;
}
```

Output:

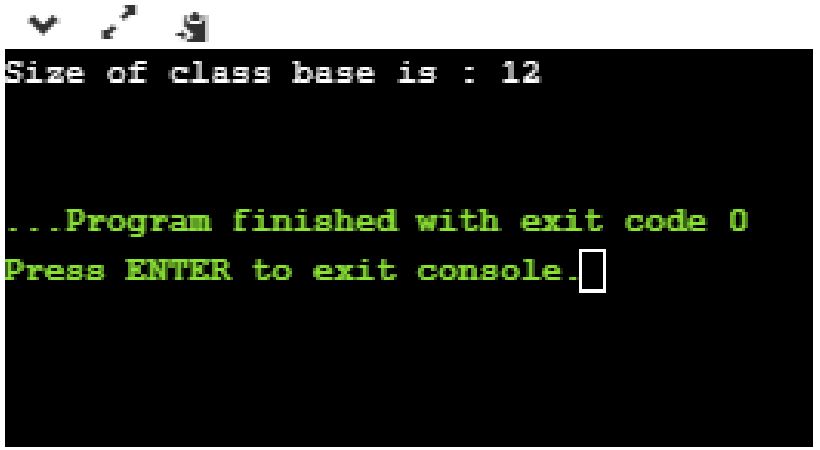


In the above code, we have added one more integer variable. In this case, the size of the class would be 8 bytes as **int** variable occupies 4 bytes, so two integer variables occupy 8 bytes.

If we add a char variable in the above code, then the code would look like:

```
#include <iostream>
using namespace std;
class Base
{
    int a;
    int d;
    char ch;
};
int main()
{
  Base b;
  cout << "Size of class base is : "<<sizeof(b) << endl;
    return 0;
}
```

Output:

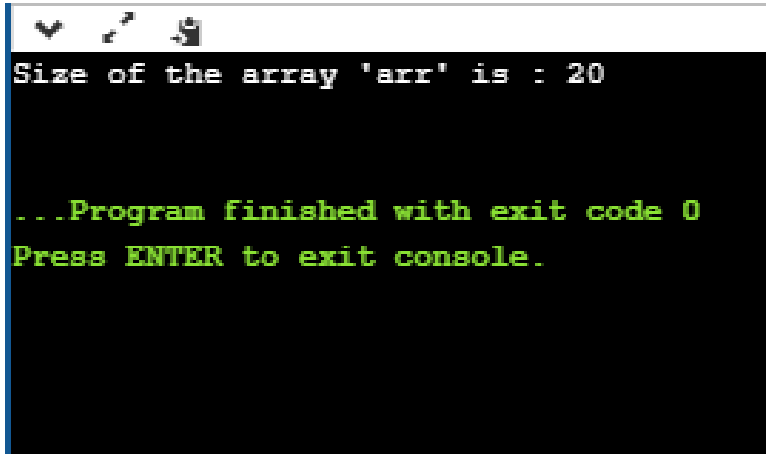


In the above code, the class has two integer variables, and one char variable. According to our calculation, the size of the class would be equal to 9 bytes (int+int+char), but this is wrong due to the concept of structure padding.

•**When an operand is of array type.**

```
#include <iostream>
using namespace std;
 int main()
{
  int arr[]={10,20,30,40,50};
  cout << "Size of the array 'arr' is : "<<sizeof(arr) << endl;
  return 0;
}
```

Output:

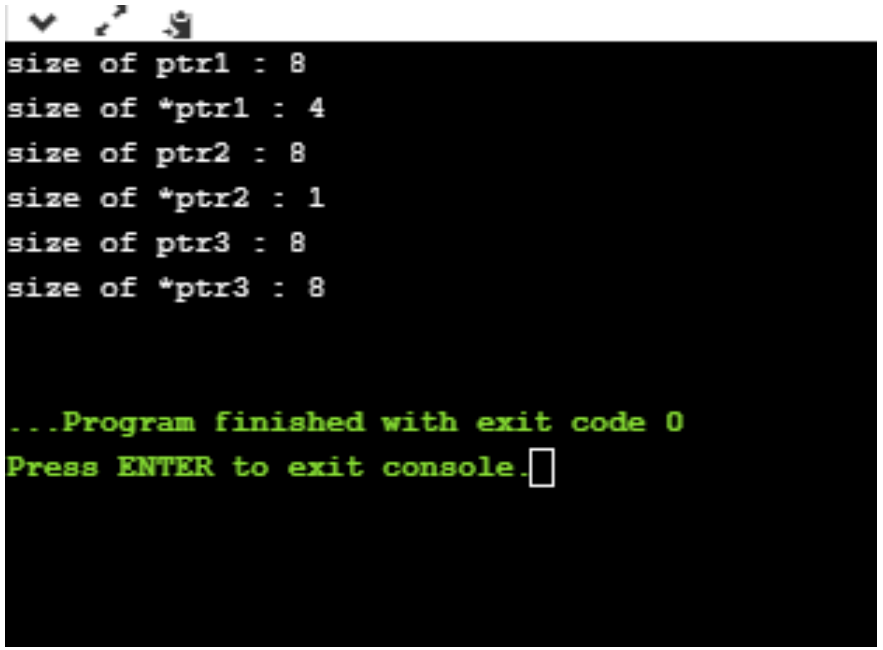


In the above program, we have declared an array of integer type which contains five elements. We have evaluated the size of the array by using **sizeof()** operator. According to our calculation, the size of the array should be 20 bytes as int data type occupies 4 bytes, and array contains 5 elements, so total memory space occupied by this array is 5*4 = 20 bytes.

•**When an operand is of pointer type.**

```
#include <iostream>
using namespace std;
int main()
{
    int *ptr1=new int(10);
    cout << "size of ptr1 : " <<sizeof(ptr1)<< endl;
    cout << "size of *ptr1 : " <<sizeof(*ptr1)<< endl;
   char *ptr2=new char('a');
   cout <<"size of ptr2 : " <<sizeof(ptr2)<< endl;
   cout <<"size of *ptr2 : "<<sizeof(*ptr2)<< endl;
   double *ptr3=new double(12.78);
    cout <<"size of ptr3 : " <<sizeof(ptr3)<< endl;
   cout <<"size of *ptr3 : "<<sizeof(*ptr3)<< endl;
    return 0;
}
```

Output:

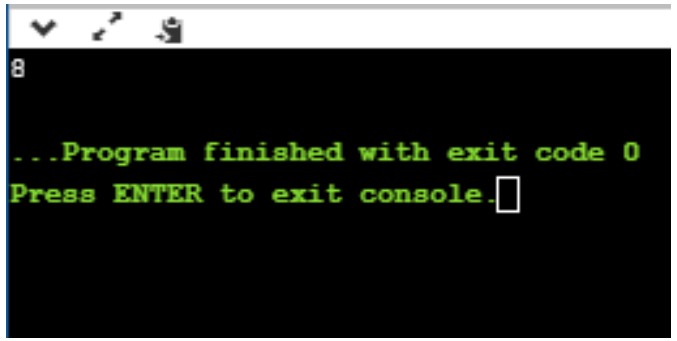


•When an operand is an expression.

```
#include <iostream>
using namespace std;

int main()
{
   int num1;
   double num2;
   cout << sizeof(num1+num2);
     return 0;
}
```

Output:



In the above program, we have declared two variables num1 and num2 of type int and double, respectively. The size of the int is 4 bytes, while the size of double is 8 bytes. The result would be the variable, which is of double type occupying 8 bytes.

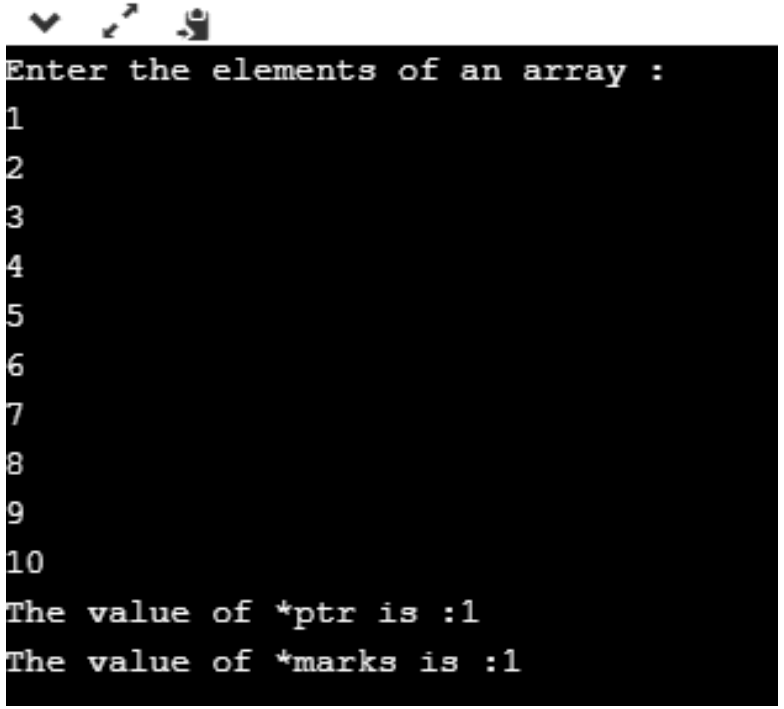
C++ Array of Pointers

Array and pointers are closely related to each other. In C++, the name of an array is considered as a pointer, i.e., the name of an array contains the address of an element. C++ considers the array name as the address of the first element. For example, if we create an array, i.e., marks which hold the 20 values of integer type, then marks will contain the address of first element, i.e., marks[0]. Therefore, we can say that array name (marks) is a pointer which is holding the address of the first element of an array.

```
#include <iostream>
using namespace std;
int main()
{
   int *ptr;  // integer pointer declaration
   int marks[10]; // marks array declaration
   cout << "Enter the elements of an array :" <<endl;
    for(int i=0;i<10;i++)
      {
         cin>>marks[i];
      }
      ptr=marks; // both marks and ptr pointing to the same element..
      cout << "The value of *ptr is :" <<*ptr<< endl;
      cout << "The value of *marks is :" <<*marks<<endl;
}
```

In the above code, we declare an integer pointer and an array of integer type. We assign the address of marks to the ptr by using the statement ptr=marks; it means that both the variables 'marks' and 'ptr' point to the same element, i.e., marks[0]. When we try to print the values of *ptr and *marks, then it comes out to be same. Hence, it is proved that the array name stores the address of the first element of an array.

Output:



Array of Pointers

An array of pointers is an array that consists of variables of pointer type, which means that the variable is a pointer addressing to some other element. Suppose we create an array of pointer holding 5 integer pointers; then its declaration would look like:

```
int *ptr[5];        // array of 5 integer pointer.
```

In the above declaration, we declare an array of pointer named as ptr, and it allocates 5 integer pointers in memory.

The element of an array of a pointer can also be initialized by assigning the address of some other element. Let's observe this case through an example.

```
int a; // variable declaration.
ptr[2] = &a;
```

In the above code, we are assigning the address of 'a' variable to the third element of an array 'ptr'.

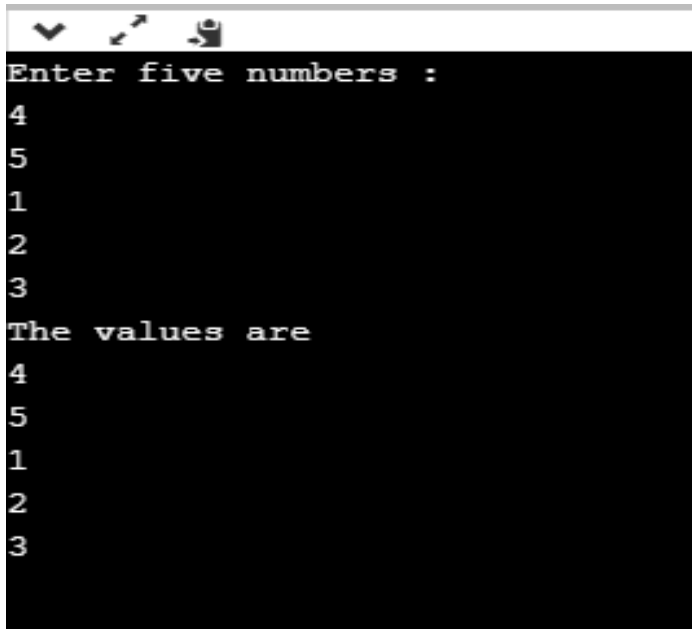
We can also retrieve the value of 'a' be dereferencing the pointer.

```
*ptr[2];
```

```cpp
#include <iostream>
using namespace std;
int main()
{
   int ptr1[5]; // integer array declaration
   int *ptr2[5]; // integer array of pointer declaration
   cout << "Enter five numbers :" << endl;
   for(int i=0;i<5;i++)
   {
      cin >> ptr1[i];
   }
   for(int i=0;i<5;i++)
   {
      ptr2[i]=&ptr1[i];
   }
   // printing the values of ptr1 array
   cout << "The values are" << endl;
   for(int i=0;i<5;i++)
   {
      cout << *ptr2[i] << endl;
   }
   }
```

In the above code, we declare an array of integer type and an array of integer pointers. We have defined the 'for' loop, which iterates through the elements of an array 'ptr1', and on each iteration, the address of element of ptr1 at index 'i' gets stored in the ptr2 at index 'i'.

Output:



Array of Pointer to Strings

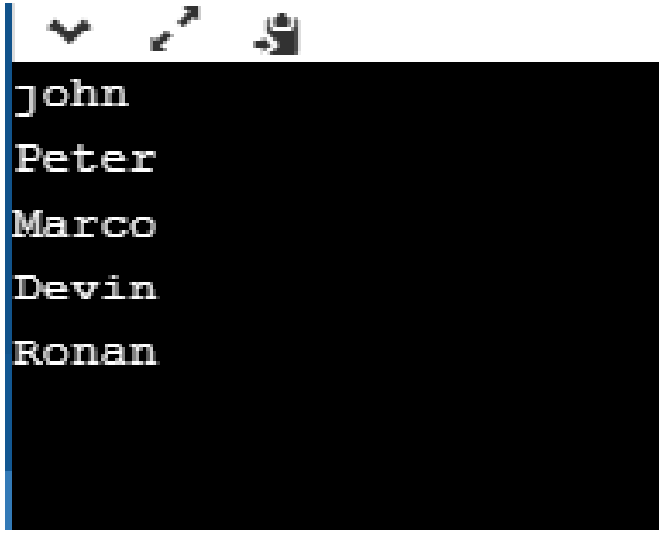
An array of pointer to strings is an array of character pointers that holds the address of the first character of a string or we can say the base address of a string.

The following are the differences between an array of pointers to string and two-dimensional array of characters:

- An array of pointers to string is more efficient than the two-dimensional array of characters in case of memory consumption because an array of pointer to strings consumes less memory than the two-dimensional array of characters to store the strings.
- In an array of pointers, the manipulation of strings is comparatively easier than in the case of 2d array. We can also easily change the position of the strings by using the pointers.

```
#include <iostream>
using namespace std;
int main()
{
  char *names[5] = {"john",
                    "Peter",
                    "Marco",
                    "Devin",
                    "Ronan"};
for(int i=0;i<5;i++)
    {
        std::cout << names[i] << std::endl;
    }
    return 0;
}
```

Output:



In the above code, we have declared an array of char pointer holding 5 string literals, and the first character of each string is holding the base address of the string.

C++ Void Pointer

A void pointer is a general-purpose pointer that can hold the address of any data type, but it is not associated with any data type.

Syntax of void pointer

```
void *ptr;
```

In C++, we cannot assign the address of a variable to the variable of a different data type. Consider the following example:

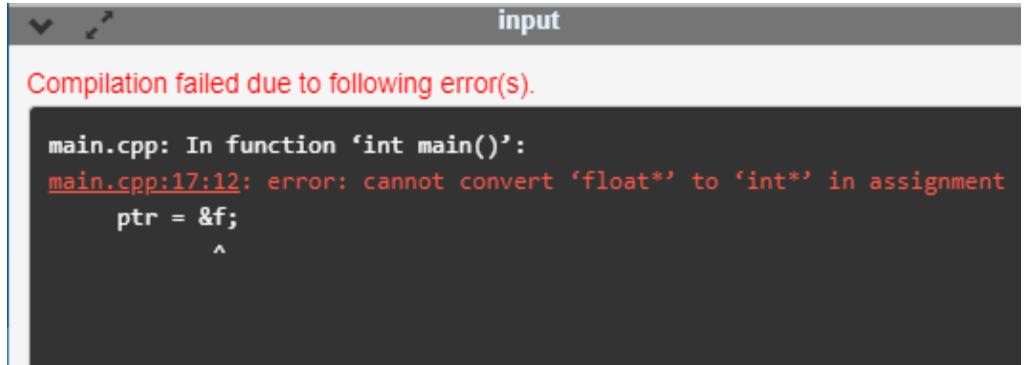
```
int *ptr;  // integer pointer declaration
float a=10.2; // floating variable initialization
ptr= &a;  // This statement throws an error.
```

In the above example, we declare a pointer of type integer, i.e., ptr and a float variable, i.e., 'a'. After declaration, we try to store the address of 'a' variable in 'ptr', but this is not possible in C++ as the variable cannot hold the address of different data types.

```
#include <iostream.h>
using namespace std;
int main()
{
    int *ptr;
    float f=10.3;
    ptr = &f; // error
    cout << "The value of *ptr is : " <<*ptr<< endl;
    return 0;
}
```

In the above program, we declare a pointer of integer type and variable of float type. An integer pointer variable cannot point to the float variable, but it can point to an only integer variable.

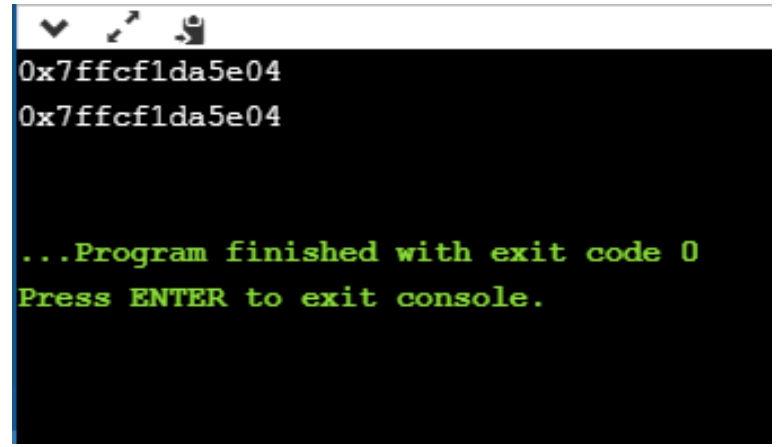
Output:



C++ has overcome the above problem by using the C++ void pointer as a void pointer can hold the address of any data type.

Let's look at a simple example of void pointer.

```
#include <iostream>
using namespace std;
int main()
{
  void *ptr;   // void pointer declaration
  int a=9;   // integer variable initialization
  ptr=&a;   // storing the address of 'a' variable in a void pointer variable.
  std::cout << &a << std::endl;
  std::cout << ptr << std::endl;
  return 0;
}
```



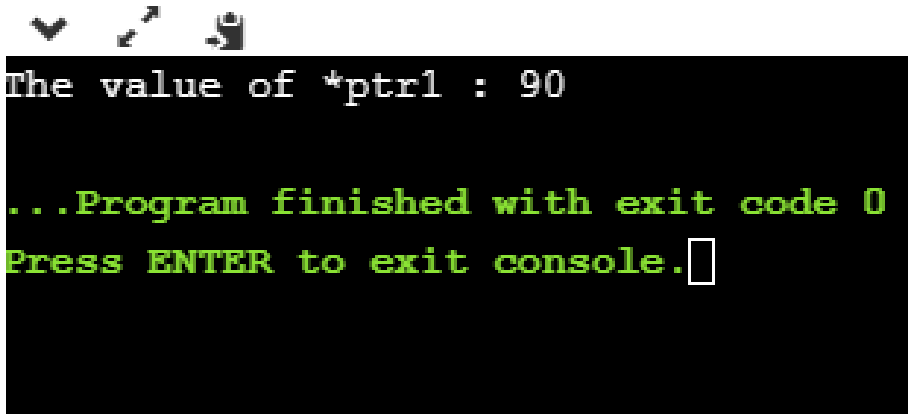
In the above program, we declare a void pointer variable and an integer variable where the void pointer contains the address of an integer variable.

Difference between void pointer in C and C++

In C, we can assign the void pointer to any other pointer type without any typecasting, whereas in C++, we need to typecast when we assign the void pointer type to any other pointer type.

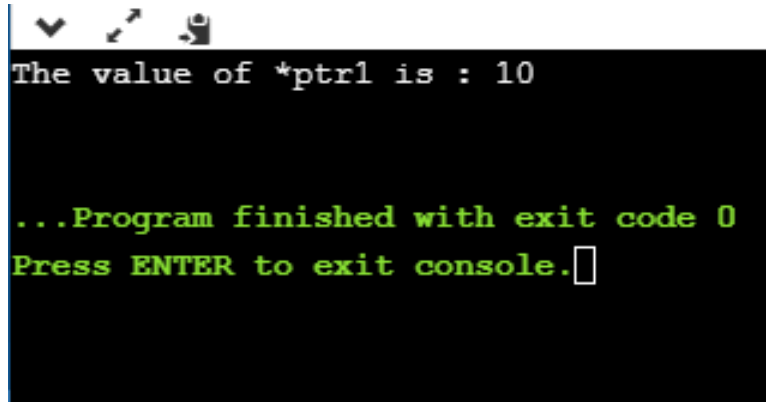
In C,

```
#include <stdio.h>
int main()
{
   void *ptr; // void pointer declaration
   int *ptr1;  // integer pointer declaration
   int a =90;  // integer variable initialization
   ptr=&a; // storing the address of 'a' in ptr
   ptr1=ptr; // assigning void pointer to integer pointer type.
   printf("The value of *ptr1 : %d",*ptr1);
   return 0;
}
```



In C++,

```
#include <iostream>
using namespace std;
int main()
{
  void *ptr; // void pointer declaration
  int *ptr1; // integer pointer declaration
  int data=10; // integer variable initialization
  ptr=&data;  // storing the address of data variable in void pointer variable
  ptr1=(int *)ptr; // assigning void pointer to integer pointer
  cout << "The value of *ptr1 is : " <<*ptr1<< endl;
  return 0;
}
```



C++ References

Till now, we have read that C++ supports two types of variables:

- An ordinary variable is a variable that contains the value of some type. For example, we create a variable of type int, which means that the variable can hold the value of type integer.
- A pointer is a variable that stores the address of another variable. It can be dereferenced to retrieve the value to which this pointer points to.
- There is another variable that C++ supports, i.e., references. It is a variable that behaves as an alias for another variable.

How to create a reference?

Reference can be created by simply using an ampersand (&) operator. When we create a variable, then it occupies some memory location. We can create a reference of the variable; therefore, we can access the original variable by using either name of the variable or reference. For example,

int a=10;

Now, we create the reference variable of the above variable.

int &ref=a;

The above statement means that 'ref' is a reference variable of 'a', i.e., we can use the 'ref' variable in place of 'a' variable.

C++ provides two types of references:

- References to non-const values
- References as aliases

References to non-const values

It can be declared by using & operator with the reference type variable.

```
#include <iostream>
using namespace std;
int main()
{
int a=10;
int &value=a;
cout << value << endl;
return 0;
}
```

Output: 10

References as aliases

References as aliases is another name of the variable which is being referenced.

For example,

```
int a=10;   // 'a' is a variable.
int &b=a; // 'b' reference to a.
int &c=a; // 'c' reference to a.
```

Let's look at a simple example.

```
#include <iostream>
using namespace std;
int main()
{
int a=70; // variable initialization
int &b=a;
int &c=a;
cout << "Value of a is :" <<a<< endl;
cout << "Value of b is :" <<b<< endl;
cout << "Value of c is :" <<c<< endl;
return 0;}
```

Output:

```
Value of a is :70
Value of b is :70
Value of c is :70
```

Properties of References

The following are the properties of references:

Initialization:

It must be initialized at the time of the declaration.

```
#include <iostream>
using namespace std;
int main()
{
int a=10; // variable initialization
int &b=a; // b reference to a
cout << "value of a is " <<b<< endl;
return 0;
}
```

Output:

value of a is 10

In the above code, we have created a reference variable, i.e., 'b'. At the time of declaration, 'a' variable is assigned to 'b'. If we do not assign at the time of declaration, then the code would look like:

```
int &b;
&b=a;
```

The above code will throw a compile-time error as 'a' is not assigned at the time of declaration.

Reassignment

It cannot be reassigned means that the reference variable cannot be modified.

```
#include <iostream>
using namespace std;
int main()
{
int x=11; // variable initialization
int z=67;
int &y=x; // y reference to x
int &y=z; // y reference to z, but throws a compile-time error.
return 0;}
```

In the above code, 'y' reference variable is referring to 'x' variable, and then 'z' is assigned to 'y'. But this reassignment is not possible with the reference variable, so it throws a compile-time error.

Function Parameters

References can also be passed as a function parameter. It does not create a copy of the argument and behaves as an alias for a parameter. It enhances the performance as it does not create a copy of the argument.

Let's understand through a simple example.

```
#include <iostream>
using namespace std;
int main()
{
int a=9; // variable initialization
int b=10; // variable initialization
swap(a, b); // function calling
cout << "value of a is :" <<a<< endl;
cout << "value of b is :" <<b<< endl;
return 0;
}
```

```
void swap(int &p, int &q) // function definition
{
int temp; // variable declaration
temp=p;
p=q;
q=temp;
}
```

Output:

```
value of a is :10
value of b is :9
```

References as shortcuts

With the help of references, we can easily access the nested data.

```
#include <iostream>
using namespace std;
struct profile
{
int id;
};
struct employee
{
profile p;
};
int main()
{
employee e;
int &ref=e.p.id;
ref=34;
std::cout << e.p.id << std::endl;
}
```

In the above code, we are trying to access the 'id' of the profile struct of the employee. We generally access this member by using the statement e.p.id, but this would be a tedious task if we have multiple access to this member. To avoid this situation, we create a reference variable, i.e., ref, which is another name of 'e.p.id'.

Output

34

Differences between Reference and Pointer

	References	Pointers
Definition	A **reference** is an alias for an already existing variable. Once a reference is initialized to a variable, it cannot be changed to refer to another variable.	A **pointer** in C++ is a variable that holds the memory address of another variable.

Reassignment	It cannot be re-assigned to any other variable after its initialization.	After declaration, it can be re-assigned to any other variable of the same type.
		int a = 5;
		int *p;
		p = &a;
		int b = 6;
		p = &b;

Memory Address	It shares the same address as the original variable.	Pointers have their own memory address.

	References	Pointers

Initialization	It is necessary to initialize it with a value during declaration.
	int a = 5;
	int &ref = a;
		It is not necessary to initialize it with a value during declaration.
		int a = 5;
		int *p;
		p = &a;

Dereferenced	It does not need to be dereferenced and can be used simply by name.	It must be dereferenced with a * to access the variable’s value..
Arithmetic Operations	Arithmetic operations can not be performed on it.	Arithmetic operations can be performed on it.
Work	It is referring to another variable.	It is storing the address of the variable.
Null Value	It does not have null value.	It can have value assigned as null.
Arguments	This variable is referenced by the method pass by value.	The pointer does it work by the method known as pass by reference.

C++ Functions

The function in C++ language is also known as procedure or subroutine in other programming languages.
To perform any task, we can create function. A function can be called many times. It provides modularity and code reusability.

Advantage of functions in C++

1) Code Reusability

By creating functions in C++, you can call it many times. So we don't need to write the same code again and again.

2) Code optimization

It makes the code optimized, we don't need to write much code.
Suppose, you have to check 3 numbers (531, 883 and 781) whether it is prime number or not. Without using function, you need to write the prime number logic 3 times. So, there is repetition of code. But if you use functions, you need to write the logic only once and you can reuse it several times.

Function

- A function is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- Functions are used to perform certain actions, and they are important for reusing code: Define the code once, and use it many times.
- C++ provides some pre-defined functions, such as main(), which is used to execute code. But you can also create your own functions to perform certain actions.
- To create (often referred to as declare) a function, specify the name of the function, followed by parentheses ():

```
void myFunction() {
  // code to be executed
}
```

Types of Functions

There are two types of functions in C programming:

1. Library Functions: are the functions which are declared in the C++ header files such as printf(), scanf()etc.

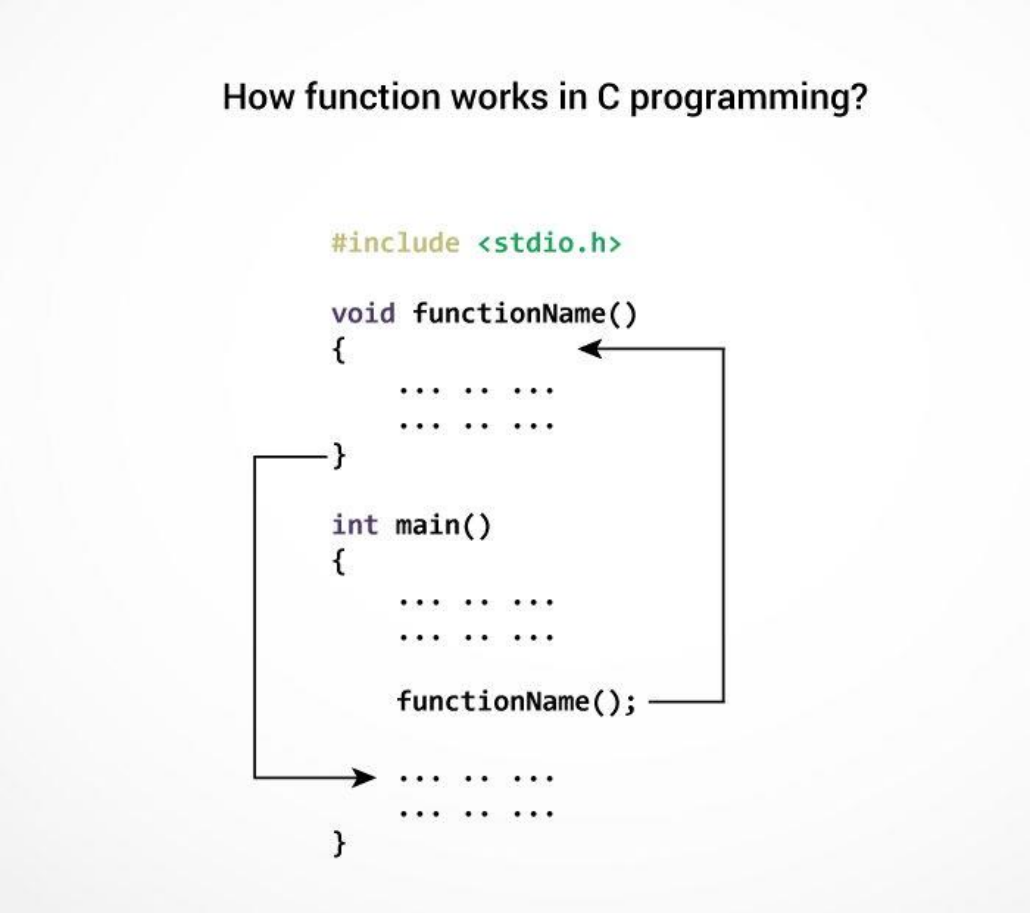
2. User-defined functions: are the functions which are created by the C++ programmer, so that he/she can use it many times. It reduces complexity of a big program and optimizes the code.

Declaration of a function

The syntax of creating function in C++ language is given below:

```
return_type  function_name(data_type parameter)
{
//code to be executed
}
```

Function


How function works in C programming?
#include <stdio.h>
void functionName()
{
... .
... .
}
int main()
{
... .
... .
functionName();
... .
... .
}

Function

```
// Create a function
void myFunction() {
  cout << "I just got executed!";
}

int main() {
  myFunction(); // call the function
  return 0;
}

// Outputs "I just got executed!"
```

Function

```
void myFunction() {
  cout << "I just got executed!\n";
}

int main() {
  myFunction();
  myFunction();
  myFunction();
  return 0;
}

// I just got executed!
// I just got executed!
// I just got executed!
```

```
void myFunction(string country = "Norway") {
  cout << country << "\n";
}

int main() {
  myFunction("Sweden");
  myFunction("India");
  myFunction();
  myFunction("USA");
  return 0;
}

// Sweden
// India
// Norway
// USA
```

Factorial

```c
#include <stdio.h>

int factorial( int n) {
    int fact = 1, i;

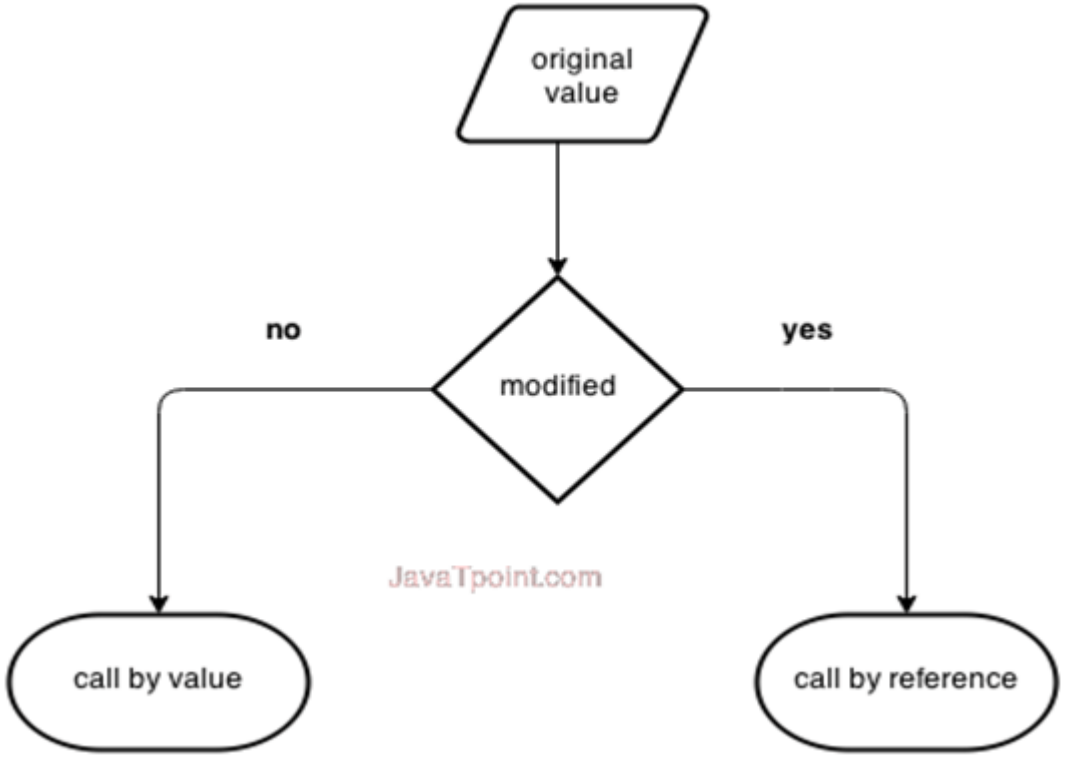
    // Loop from 1 to N to get the factorial
    for (i = 1; i <= n; i++) {
        fact = fact*i;
    }
        printf("Factorial= %d", fact);

}

int main() {
    int N = 5;
    factorial(N);
    return 0;
}
```

Call by value and call by reference in C++

There are two ways to pass value or data to function in C language: call by value and call by reference. Original value is not modified in call by value but it is modified in call by reference.



Call by value in C++

In call by value, **original value is not modified.**
In call by value, value being passed to the function is locally stored by the function parameter in stack memory location. If you change the value of function parameter, it is changed for the current function only. It will not change the value of variable inside the caller method such as main().

- There are two copies of parameters stored in different memory locations.
- One is the original copy and the other is the function copy.
- Any changes made inside functions are not reflected in the actual parameters of the caller.

Call by value in C

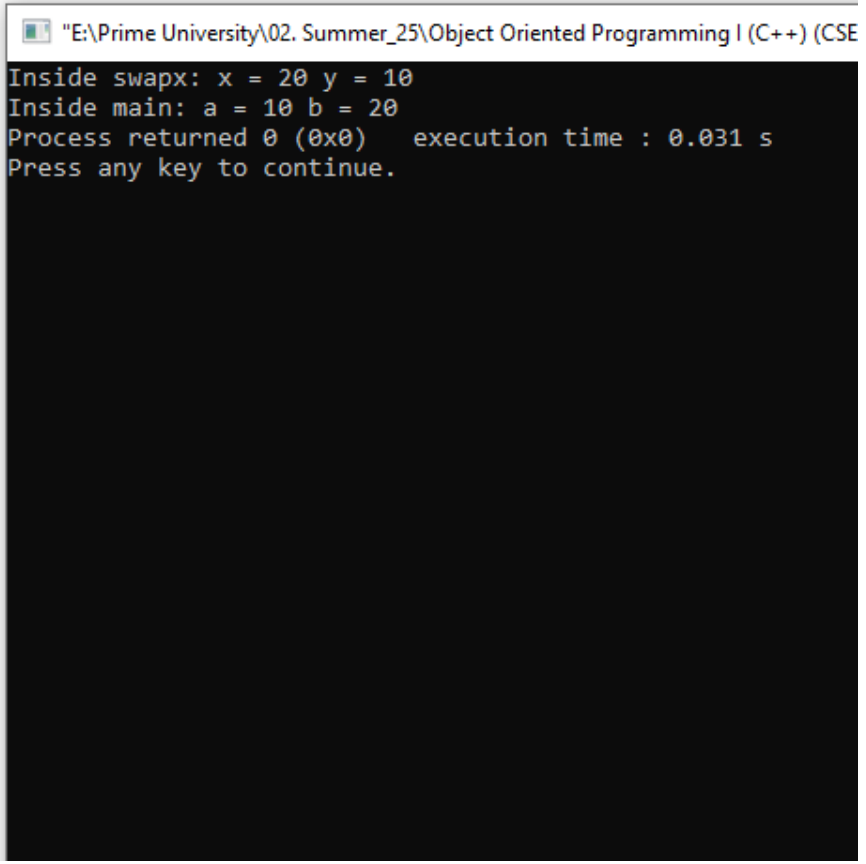
```c
#include <stdio.h>

void valueSwap(int x, int y) {
    int t;
    t = x;
    x = y;
    y = t;

    printf("Inside swapx: x = %d y = %d\n", x, y);
}

// Main function
int main() {
    int a = 10, b = 20;

    // Pass by Values
    valueSwap(a, b);
    printf("Inside main: a = %d b = %d", a, b);
    return 0;
}
```



Call by value in C++

```cpp
#include <iostream>
using namespace std;

void valueSwap(int x, int y) {
    int temp;

    temp = x;
    x = y;
    y = temp;

    cout << "Inside swap: x =" << x <<"Inside swap: y =" << y << endl;
}

// Main function
int main() {
    int a = 10, b = 20;

    // Pass by Values
    valueSwap(a, b);
    cout << "Inside main: a =" << a << "main swap: b =" << b;
    return 0;
}
```

Call by reference in C++

In call by reference, original value is modified because we pass reference (address). Here, address of the value is passed in the function, so actual and formal arguments share the same address space. Hence, value changed inside the function, is reflected inside as well as outside the function.

```
#include<iostream>
using namespace std;
void swap(int *x, int *y)
{
 int swap;
 swap=*x;
 *x=*y;
 *y=swap;
}
```

```
int main()
{
 int x=500, y=100;
 swap(&x, &y);  // passing value to function
 cout<<"Value of x is: "<<x<<endl;
 cout<<"Value of y is: "<<y<<endl;
 return 0;
}
```

Call by reference in C++

- Both the actual and formal parameters refer to the same locations.
- Any changes made inside the function are actually reflected in the actual parameters of the caller

```
#include <stdio.h>

// Function to swap two variables
// by references
void swapx(int* x, int* y) {
    int t;
    t = *x;
    *x = *y;
    *y = t;
    printf("Inside swapx: x = %d y = %d\n", *x, *y);
}

int main() {
    int a = 10, b = 20;

    // Pass by reference
    swapx(&a, &b);

    printf("Inside main: a = %d b = %d", a, b);
    return 0;
}
```

Call by value vs Call by reference

Call By Value	Call By Reference
While calling a function, we pass the values of variables to it. Such functions are known as "Call By Values".	While calling a function, instead of passing the values of variables, we pass the address of variables(location of variables) to the function known as "Call By References.
In this method, the value of each variable in the calling function is copied into corresponding dummy variables of the called function.	In this method, the address of actual variables in the calling function is copied into the dummy variables of the called function.
With this method, the changes made to the dummy variables in the called function have no effect on the values of actual variables in the calling function.	With this method, using addresses we would have access to the actual variables and hence we would be able to manipulate them.
In call-by-values, we cannot alter the values of actual variables through function calls.	In call by reference, we can alter the values of variables through function calls.
Values of variables are passed by the Simple technique.	Pointer variables are necessary to define to store the address values of variables.
This method is preferred when we have to pass some small values that should not change.	This method is preferred when we have to pass a large amount of data to the function.
Call by value is considered safer as original data is preserved	Call by reference is risky as it allows direct modification in original data

Call by value vs Call by reference

Feature	Call by Value	Call by Reference
Value Passed	In this method, the value of the variable is passed to the function.	In call by reference, reference to the variable is passed.
Scope of Changes	The original value remains unchanged even when we make changes in the function.	The changes made are reflected in the original variable.
Performance	It requires extra memory and time to copy so less efficient.	It is more memory and time efficient as it does not create a copy.
Memory Location	The memory addresses of the actual and formal parameters are different.	The actual and the formal parameters point at the same memory address.
Applications	Mainly used to pass values for small data or when we do not want to change original values.	It is used when we want to modify the original value or save resources.

Difference between call by value and call by reference in C++

No.	Call by value	Call by reference
1	A copy of value is passed to the function	An address of value is passed to the function
2	Changes made inside the function is not reflected on other functions	Changes made inside the function is reflected outside the function also
3	Actual and formal arguments will be created in different memory location	Actual and formal arguments will be created in same memory location
4.	The original value is not modified	The original value is modified

File

File handling in C is the process in which we create, open, read, write, and close operations on a file. C language provides different functions such as **fopen()**, **fwrite()**, **fread()**, **fseek()**, **fprintf()**, etc. to perform input, output, and many different C file operations in our program.

C File Operations

C language provides the following different operations that we can perform on a file from our C program:

- Creating a new file.
- Opening an existing file.
- Reading from file.
- Writing to a file.
- Moving to a specific location in a file.
- Closing a file.

Open a File in C

For opening a file in C, the **fopen()** function is used with the filename or file path along with the required **access modes**.

```
FILE* ptr;
ptr =  fopen(file_name, access_mode);
```

Parameters

- ***file_name:*** name of the file when present in the same directory as the source file. Otherwise, full path.
- ***access_mode****:* Specifies for what operation the file is being opened.

Return Value

- If the file is opened successfully, returns a file pointer to it.
- If the file is not opened, then returns NULL.

File

FILE *fptr;
fptr = fopen(filename, mode);

FILE is basically a data type, and we need to create a pointer variable to work with it (fptr). For now, this line is not important. It's just something you need when working with files.

To actually open a file, use the **fopen()** function, which takes two parameters:

Parameter	Description
filename	The name of the actual file you want to open (or create), like filename.txt
mode	A single character, which represents what you want to do with the file (read, write or append): ***w - Writes to a file*** ***a - Appends new data to a file*** ***r - Reads from a file***

Create a File

To create a file, you can use the w mode inside the **fopen()** function.

The **w mode** is used to write to a file. However, if the file does not exist, it will create one for you:

```
FILE *fptr;

// Create a file
fptr = fopen("filename.txt", "w");

// Close the file
fclose(fptr);
```

File

```c
#include <stdio.h>
#include <stdlib.h>

int main() {

    // File pointer to store the
    // value returned by fopen
    FILE* fptr;

    // Opening the file in read mode
    fptr = fopen("filename.txt", "r");

    // checking if the file is
    // opened successfully
    if (fptr == NULL) {
        printf("The file is not opened.");
    }
    return 0;
}
```

Create a File

The fopen() function can not only open a file but also can create a file if it does not exist already. For that, we have to use the modes that allow the creation of a file if not found such as **w, w+, wb, wb+, a, a+, ab, and ab+**.

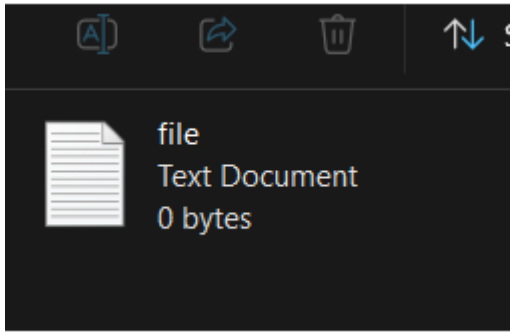
```c
#include <stdio.h>
#include <stdlib.h>

int main() {

    // File pointer
    FILE* fptr;

    // Creating file using fopen()
    // with access mode "w"
    fptr = fopen("file.txt", "w");

    // checking if the file is created
    if (fptr == NULL)
        printf("The file is not opened.");
    else
        printf("The file is created Successfully.");
    return 0;
}
```



The file is created Successfully.

C Write To Files

Let's use the w mode from the previous chapter again, and write something to the file we just created.

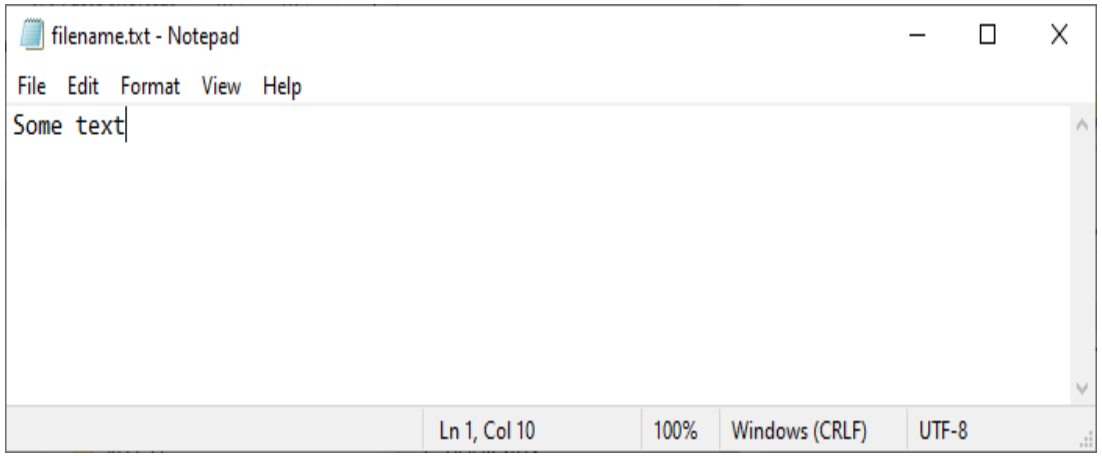
The w mode means that the file is opened for writing. To insert content to it, you can use the **fprintf()** function and add the pointer variable (**fptr** in our example) and some text:

```
FILE *fptr;

// Open a file in writing mode
fptr = fopen("filename.txt", "w");

// Write some text to the file
fprintf(fptr, "Some text");

// Close the file
fclose(fptr);
```



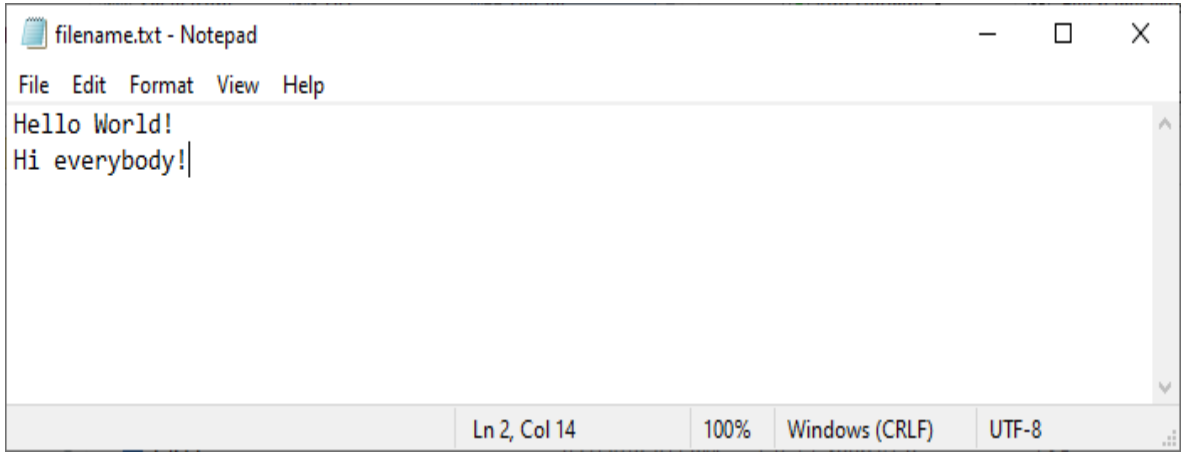
Append Content To a File

```
FILE *fptr;

// Open a file in append mode
fptr = fopen("filename.txt", "a");

// Append some text to the file
fprintf(fptr, "\nHi everybody!");

// Close the file
fclose(fptr);
```



Write to a File

The file write operations can be performed by the functions **fprintf() and fputs().** C programming also provides some other functions that can be used to write data to a file such as:

Function	Description
fprintf()	Similar to printf(), this function uses formatted string and variable arguments list to print output to the file.
fputs()	Prints the whole line in the file and a newline at the end.
fputc()	Prints a single character into the file.
fputw()	Prints a number to the file.
fwrite()	This function writes the specified number of bytes to the binary file.

Write to a File

```c
#include <stdio.h>
#include <stdlib.h>

int main() {

    // File pointer
    FILE* fptr;

    // Get the data to be written in file
    char data[50] = "GeeksforGeeks-A Computer "
                    "Science Portal for Geeks";

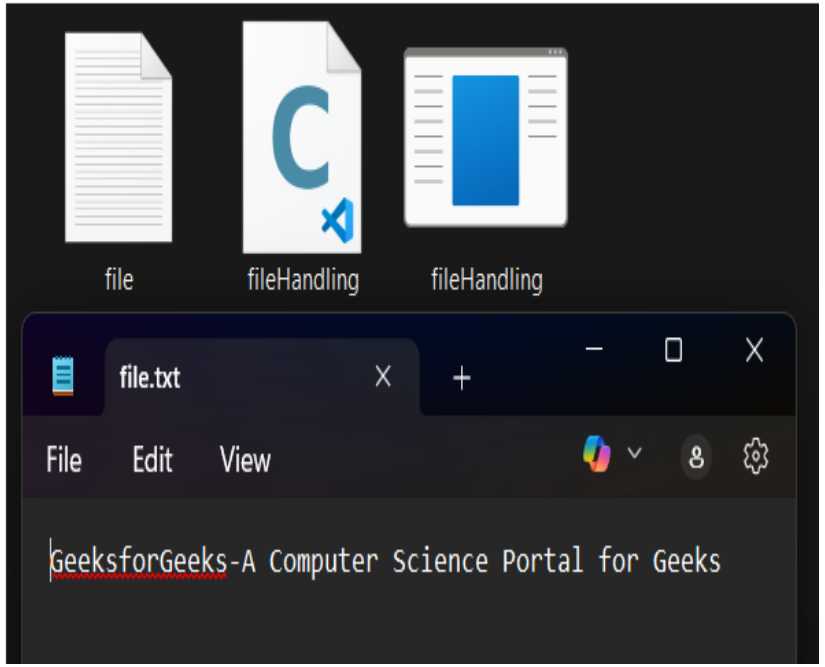
    // Creating file using fopen()
    // with access mode "w"
    fptr = fopen("file1.txt", "w");

    // Checking if the file is created
    if (fptr == NULL)
        printf("The file is not opened.");
    else{
        printf("The file is now opened.\n");
        fputs(data, fptr);
        fputs("\n", fptr);

        // Closing the file using fclose()
        fclose(fptr);
        printf("Data successfully written in file "
               "file.txt\n");
        printf("The file is now closed.");
    }
    return 0;
}
```

Output

```
The file is now opened.
Data successfully written in file file.txt
The file is now closed.
```



C Read Files

This will make the **filename.txt** opened for reading.

```
FILE *fptr;

// Open a file in read mode
fptr = fopen("filename.txt", "r");
```

Reading From a File

The file read operation in C can be performed using functions **fscanf()** or **fgets()**. Both the functions performed the same operations as that of **scanf()** and gets but with an additional parameter, the file pointer. There are also other functions we can use to read from a file. Such functions are listed below:

Function	Description
fscanf()	Use formatted string and variable arguments list to take input from a file.
fgets()	Input the whole line from the file.
fgetc()	Reads a single character from the file.
fgetw()	Reads a number from a file.
fread()	Reads the specified bytes of data from a binary file.

File

File Operations in C Programming

Open -> Write -> Read -> Close

When working with files, you need to declare a pointer of type file: FILE *fp;

File Open	File Write	File Read	File Close
fp = fopen("fileopen","mode");	fputc(int c, FILE *fp);	int c = fgetc()	fclose(fp);
File Mode: r, w, a, r+, w+, a+	fprintf(FILE *fp, sting format..)	char buff[100] fscanf(fp, "%s", buff);	
	fputs(const char *s, FILE *fp);	fgets(buff, 255, (FILE*)fp);	

File

```c
#include<stdio.h>
int main()
{
    FILE *fptr;
    fptr = fopen("prime1.txt", "r");

    int i;
    for( i=65; i < 90; i++)
    {
        fputc(i, fptr);
    }

    char array[100] = "This is CSE Depertment\n";
    fputs(array, fptr);

    fprintf(fptr, "My id is: %d\n", 100);


    int a;
    a = fgetc(fptr);
    printf("The char is: %c\n", a);

    char buff[100];
    fscanf(fptr, "%s", buff);
    printf("The data buffer is: %s\n", buff);

    fgets(buff, 500, fptr);
    printf("The data buffer is: %s\n", buff);


    fclose(fptr);
}
```

```c
#include<stdio.h>
int main()
{
   FILE *fptr;
   fptr = fopen("prime1.txt", "r");

   int i;
   for( i=65; i < 90; i++)
   {
      fputc(i, fptr);
   }
   char array[100] = "This is CSE Depertment\n";
   fputs(array, fptr);
   fprintf(fptr, "My id is: %d\n", 100);
   int a;
   a = fgetc(fptr);
   printf("The char is: %c\n", a);

   char buff[100];
   fscanf(fptr, "%s", buff);
   printf("The data buffer is: %s\n", buff);

   fgets(buff, 500, fptr);
   printf("The data buffer is: %s\n", buff);

   fclose(fptr);
}
```

Reading From a File

```c
#include <stdio.h>
#include <string.h>

int main() {
    FILE* fptr;

    // Declare the character array
    // for the data to be read from file
    char data[50];
    fptr = fopen("file.txt", "r");

    if (fptr == NULL) {
        printf("file.txt file failed to open.");
    }
    else {

        printf("The file is now opened.\n");

        // Read the data from the file
        // using fgets() method
        while (fgets(data, 50, fptr)
               != NULL) {

            // Print the data
            printf("%s", data);
        }

        // Closing the file using fclose()
        fclose(fptr);
    }
    return 0;
}
```

Output

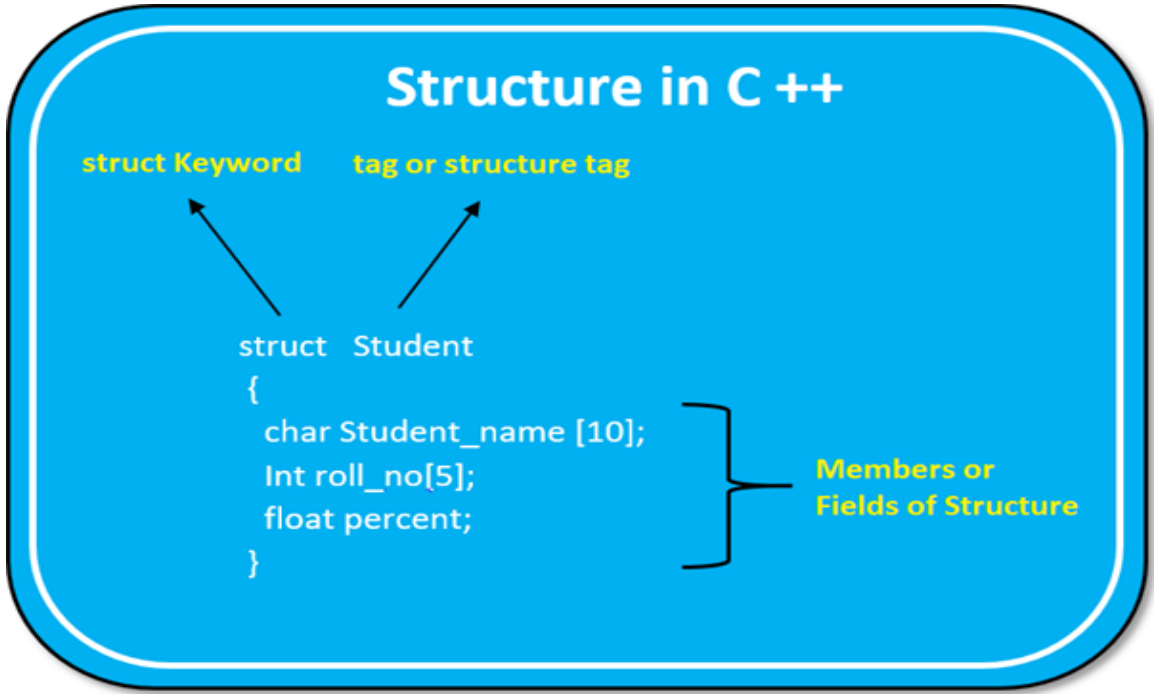
```
The file is now opened.
GeeksforGeeks-A Computer Science Portal for Geeks
```

More Functions for C File Operations

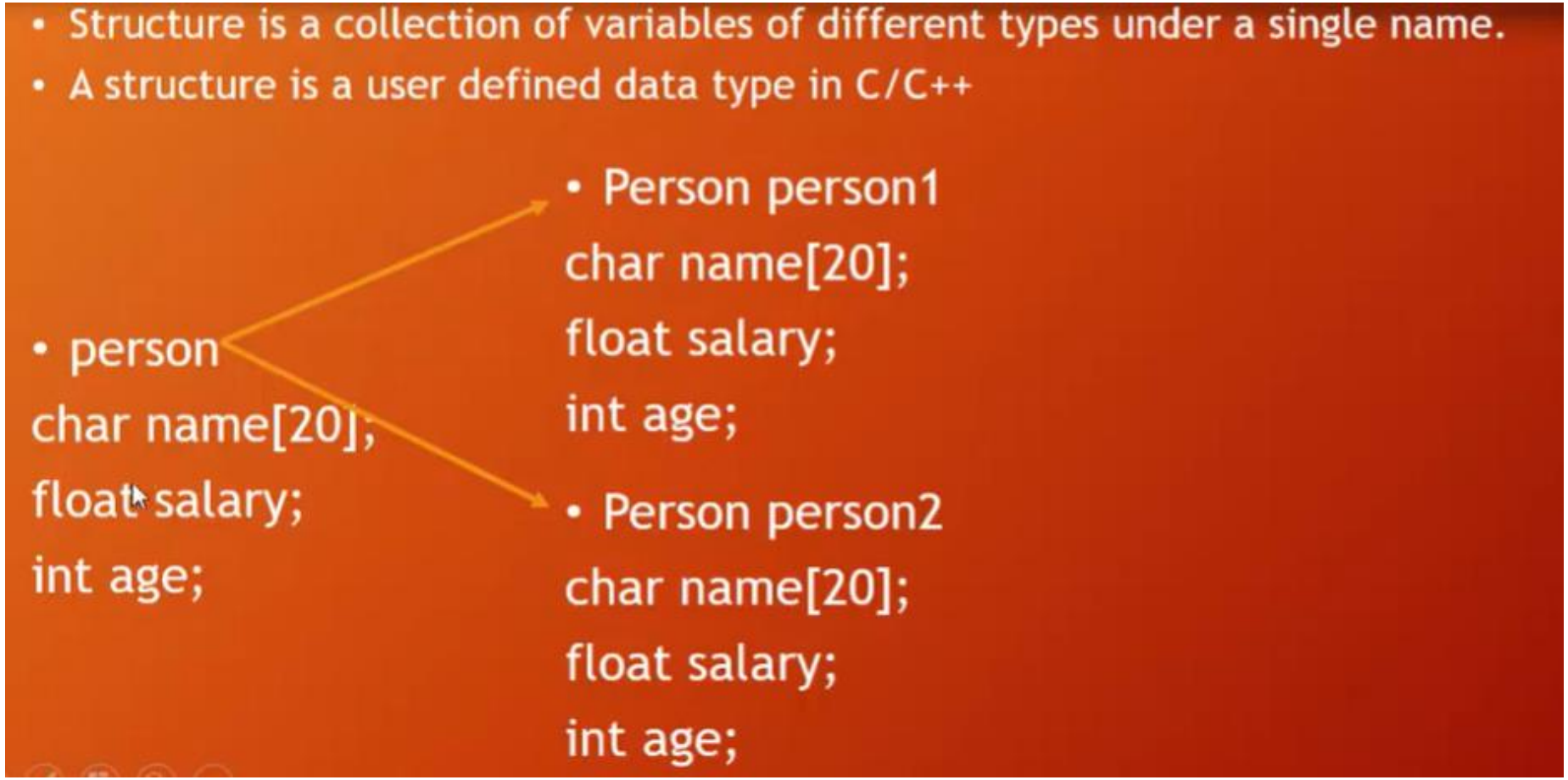
Functions	Description
fopen()	It is used to create a file or to open a file.
fclose()	It is used to close a file.
fgets()	It is used to read a file.
fprintf()	It is used to write blocks of data into a file.
fscanf()	It is used to read blocks of data from a file.
getc()	It is used to read a single character to a file.
putc()	It is used to write a single character to a file.
fseek()	It is used to set the position of a file pointer to a mentioned location.
ftell()	It is used to return the current position of a file pointer.
rewind()	It is used to set the file pointer to the beginning of a file.
putw()	It is used to write an integer to a file.
getw()	It is used to read an integer from a file.

Structure in C

In C programming, a structure (or struct) is a user-defined data type that allows for the grouping of multiple variables, potentially of different data types, under a single name. This provides a way to organize and store related data together, making it more manageable and accessible.



Structure in C



Structure

- A **structure** in C is a derived or user-defined data type. We use the keyword **struct** to define a custom data type that groups together the elements of different types. The difference between an array and a structure is that an array is a homogenous collection of similar types, whereas a structure can have elements of different types stored adjacently and identified by a name.
- We are often required to work with values of different data types having certain relationships among them. For example, a **book** is described by its **title** (string), **author** (string), **price** (double), **number of pages** (integer), etc. Instead of using four different variables, these values can be stored in a single **struct** variable.
- To access the members of a structure, first, you need to declare a structure variable and then use the **dot (.) operator** along with the structure variable.

Structure

```
struct structureName {
  dataType member1;
  dataType member2;
  ...
};

struct Person {
  char name[50];
  int citNo;
  float salary;
};
```

Create struct Variables

```
struct Person {
  // code
};
int main() {
  struct Person person1, person2, p[20];
  return 0;
}
```

Structure Example

```c
#include <stdio.h>

struct book{
    char title[10];
    char author[20];
    double price;
    int pages;
};

int main(){
    struct book book1 = {"Learn C", "Dennis Ritchie", 675.50, 325};

    printf("Title:  %s \n", book1.title);
    printf("Author: %s \n", book1.author);
    printf("Price:  %lf\n", book1.price);
    printf("Pages:  %d \n", book1.pages);
    return 0;
}
```

Structure Example

```c
#include <stdio.h>
#include <string.h>

struct Person {
   char name[50];
   int citNo;
   float salary;
}person1;

int main() {

      //struct Person person1;

   strcpy(person1.name, "George Orwell");
   person1.citNo = 1984;
   person1. salary = 2500;

   printf("Name: %s\n", person1.name);
   printf("Citizenship No.: %d\n", person1.citNo);
   printf("Salary: %.2f", person1.salary);

   return 0;
}
```

Structure Example

```c
#include <stdio.h>
#include <string.h>

struct Person {
   char name[50];
   int citNo;
   float salary;
};

int main() {

      struct Person person1;

   strcpy(person1.name, "George Orwell");
   person1.citNo = 1984;
   person1. salary = 2500;

   printf("Name: %s\n", person1.name);
   printf("Citizenship No.: %d\n", person1.citNo);
   printf("Salary: %.2f", person1.salary);

   return 0;
}
```

C++ Recursion

When function is called within the same function, it is known as recursion in C++. The function which calls the same function, is known as recursive function.

A function that calls itself, and doesn't perform any task after function call, is known as tail recursion. In tail recursion, we generally call the same function with return statement.

```
recursionfunction()
{
recursionfunction(); //calling self function
}
```

C++ Recursion Example

```
#include<iostream>
using namespace std;
int main()
{
int factorial(int);
int fact, value;
cout<<"Enter any number: ";
cin>>value;
fact=factorial(value);
cout<<"Factorial of a number is: "<<fact<<endl;
return 0;
}
```

```
int factorial(int n)
{
if(n<0)
return(-1); /*Wrong value*/
if(n==0)
return(1);  /*Terminating condition*/
else
{
return(n*factorial(n-1));
}
}
```

We can understand the above program of recursive method call by the figure given below:

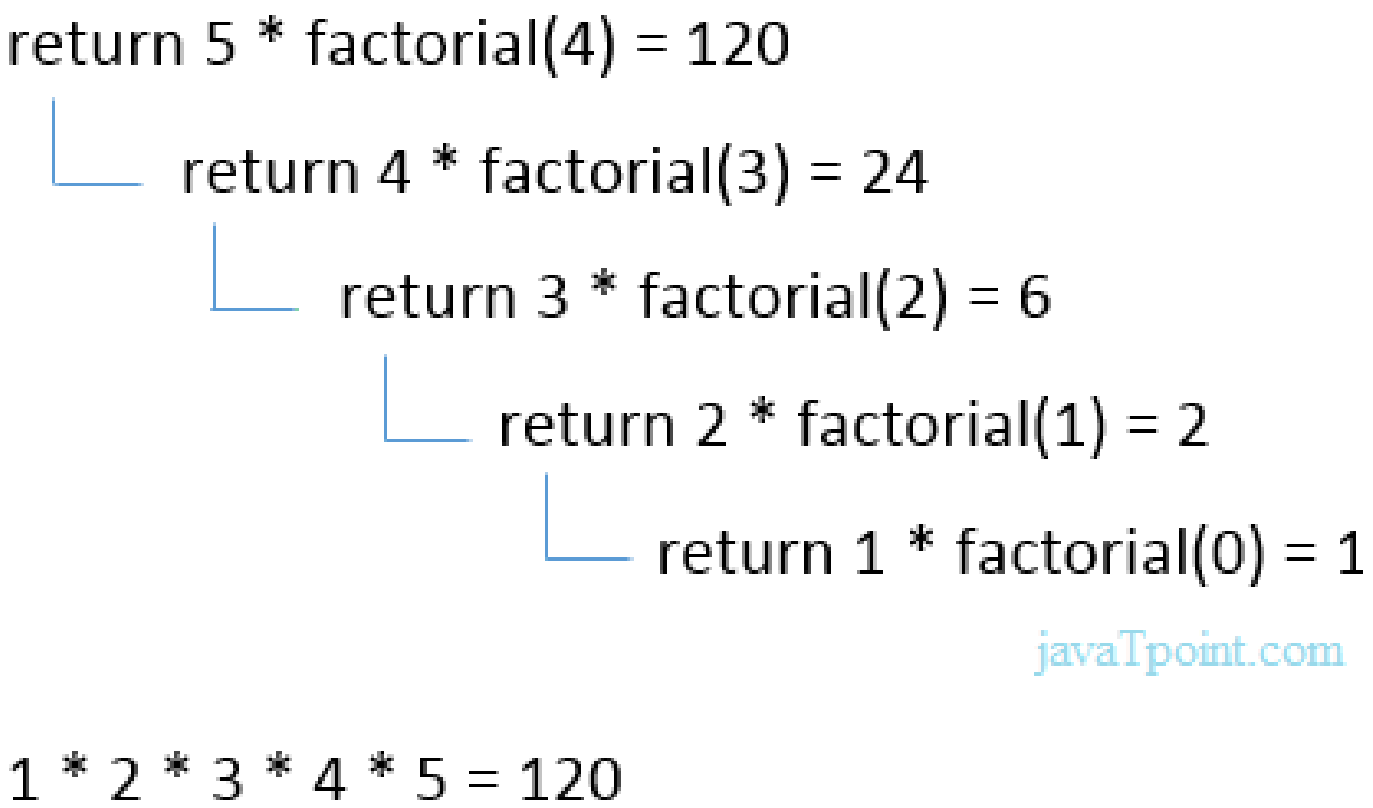


Fig: Recursion